Efficient Transformer Architectures and Beyond the Transformer

March 2026

Abstract

This note studies efficient Transformer variants and sequence architectures beyond the Transformer through one shared question: *how should a model represent and update the prefix it has already seen?* The main text moves from exact attention and cache growth to sparse retrieval, low-rank summaries, linear attention, recurrent state updates, state-space models, and hybrid memory designs. The central practical lesson is that asymptotic complexity, cache design, and kernel implementation have to be judged together: architectural ideas only matter if they improve the actual bottleneck on real hardware.

How this note is organized

This note is intentionally layered so it works both as a first pass and as a technical reference.

1. **Core ideas.** The main path emphasizes intuition, repeated mental models, toy examples, and short derivations.
2. **Architecture survey.** The middle sections compare efficient attention, recurrent models, state-space models, and hybrid designs.
3. **Technical appendices.** The appendices preserve fuller derivations and broader historical notes for a second reading.

The central question behind the field

Every architecture in this note answers the same question:

$$\textit{How should a sequence model remember the prefix } x_{<t}\textit{?}$$

There are only a few broad answers. A model can keep all past tokens and retrieve them exactly, keep only selected or compressed summaries, compress the whole past into a recurrent state, apply a learned convolutional filter to the past, or combine several of these ideas in a hybrid design. Once this question becomes familiar, the literature stops looking like a disconnected list of papers and starts looking like a small number of recurring design patterns.

Contents

1 How to Read This Document

Three reading routes

Route 1: first-time reader. Read Sections 2–9, do the worked examples, and skip the appendices on the first pass.
Route 2: graduate course / self-study. Read the main text and then use Appendices A–E as derivation notes.
Route 3: researcher refresher. Skim Sections 7–8 first, then jump into the appendices for whichever branch you need in detail.

What "efficient" means here. Efficiency is not a single number. Throughout the document we distinguish:

- sequence-length complexity during training;
- real hardware efficiency on GPUs/TPUs;
- autoregressive inference KV-cache size or recurrent-state size;
- latency and throughput at long context;
- model quality on standard language modeling, long-context retrieval, and extrapolation.

A method can have better asymptotic complexity but still lose in practice if its kernels are unfriendly to the hardware. This is why exact-attention engineering, especially FlashAttention, remains important even in a survey about alternatives to attention.

What this document covers. The literature splits naturally into two overlapping programs.

1. **Efficient Transformers:** keep the Transformer template but make attention cheaper or the cache smaller.
2. **Architectures beyond the Transformer:** replace some or all attention layers by recurrence, convolution, state-space dynamics, or explicit memory.

The rest of the document shows that these programs are not separate worlds. They meet in a common language of *prefix representation*: what is stored, how it is updated, and how it is read.

2 The Whole Field in One Picture

Readers often first meet this field as a long list of model names. That is the hardest possible way to learn it. A much easier way is to remember Figure 1. The field is about a single design decision: *what representation of the past should the model carry forward?*

Four memory styles.

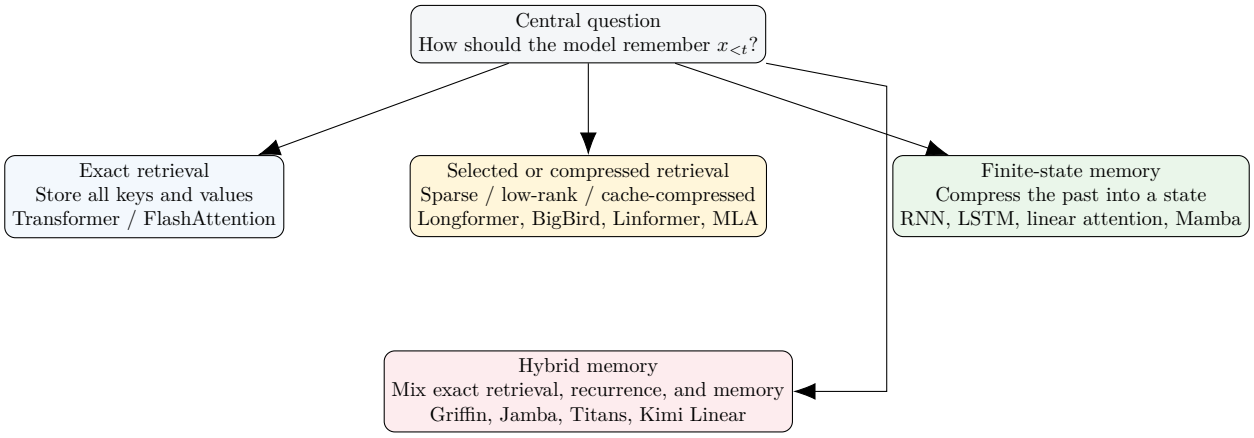

Figure 1: A compact map of the field. Most methods fit one of these four memory choices.

- **Exact retrieval.** Keep a KV cache of past keys and values and let each new query look back at all of them. This is the standard Transformer view.
- **Selected or compressed retrieval.** Still use attention-like retrieval, but only over a sparse pattern, a low-rank summary, or a compressed latent cache.
- **Finite-state memory.** Replace the growing cache by a state of fixed size. The update can be additive, gated, corrective, or selective.
- **Hybrid memory.** Keep a small amount of exact retrieval where it is most useful and combine it with recurrent or explicit long-term memory elsewhere.

The toy memory task used throughout the survey

We repeatedly reuse the same tiny example. Imagine two past key-value pairs

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

A later query

$$q = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

should recover something close to v_1. We will ask the same question again and again:

> How does each architecture store the two past associations, and how does it read the one that matches the query?

Keeping one running example removes a lot of mystery. The formulas change across model families, but the underlying task does not.

Worked example 1: exact attention on the toy task

Take the unnormalized dot-product scores

$$q^\top k_1 = 2, \qquad q^\top k_2 = 0.$$

After the softmax,

$$\alpha_1 = \frac{e^2}{e^2+1} \approx 0.88, \qquad \alpha_2 = \frac{1}{e^2+1} \approx 0.12.$$

So exact attention returns

$$y \approx 0.88\, v_1 + 0.12\, v_2.$$

The answer is close to v_1 because the query is much more aligned with k_1 than with k_2. This is the fundamental attention story: compare the query with each stored key, convert the scores into weights, and form a weighted average of the stored values.

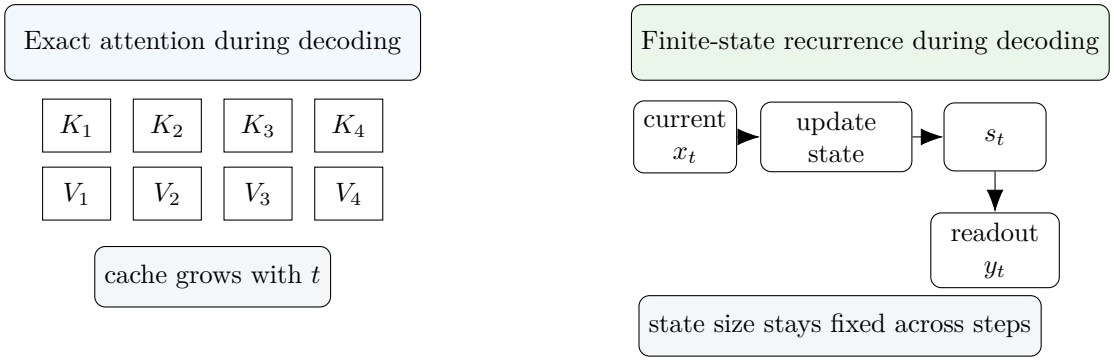

Figure 2: The most important systems distinction for inference. Transformer decoding grows a KV cache with sequence length; recurrent models keep a fixed-size state.

The first mental checkpoint

Before learning any named architecture, ask two concrete questions.

1. **What is stored?** All past tokens, a sparse subset, a low-rank summary, a recurrent state, or an explicit memory bank?
2. **How is it read?** Similarity search, a fixed filter, a state readout, or a hybrid of these?

If you can answer those two questions, the architecture is already mostly demystified.

3 Core Ideas: Language Modeling, Attention, and the Standard Transformer

3.1 The language-modeling objective

Let the token sequence be $x_{1:n} = (x_1, \ldots, x_n)$. In a causal language model, token x_t is predicted from the prefix $x_{<t}$. The loss is

$$\mathcal{L}(\theta) = -\sum_{t=1}^{n} \log p_\theta(x_t \mid x_{<t}). \tag{1}$$

This formula is simple, but it drives the whole survey. Every architecture here is a different answer to one practical question:

> How should the model summarize $x_{<t}$ so that the next-token predictor is accurate and efficient?

Table 1: Minimum notation and shapes for a first reading.

Symbol	Meaning	Typical shape
X	input token representations	$n \times d_{\text{model}}$
Q, K, V	queries, keys, values	$n \times d_k,\ n \times d_k,\ n \times d_v$
q_t, k_t, v_t	one token's query, key, value	$d_k,\ d_k,\ d_v$
S_t	matrix-valued recurrent memory	$m \times d_v$
h_t	vector-valued recurrent state	N
KV cache	stored past keys and values	grows with t

3.2 Embeddings, residual connections, layer normalization, and the MLP block

A **token embedding** turns a discrete token ID into a vector in $\mathbb{R}^{d_{\text{model}}}$. You can think of this as a learned lookup table: each token points to a row of a matrix.

A standard decoder block then alternates two sublayers:

$$\text{attention sublayer} \quad \longrightarrow \quad \text{MLP sublayer.}$$

Both are wrapped in a residual connection. If h is the input to a sublayer and $f(h)$ is the sublayer output, then the residual form is roughly

$$h \leftarrow h + f(h).$$

Residual connections are useful because they let each block make a correction rather than having to rebuild the representation from scratch.

Layer normalization rescales and recenters activations so optimization stays stable. The MLP block is a tokenwise nonlinear map that mixes coordinates inside each token representation, while attention mixes information *across* tokens.

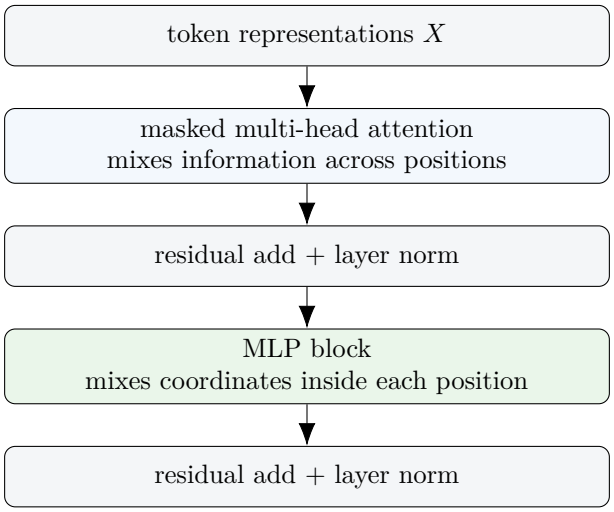

Figure 3: A standard decoder block in the simplest possible picture.

Worked example 2: what the two sublayers do

Suppose token 7 currently has a representation saying "this looks like a pronoun" and token 3 looks like the most likely antecedent. The attention sublayer can copy information from token 3 into token 7. The MLP sublayer can then transform that mixed representation into something more directly useful for the next-token prediction. In short:

- attention moves information across positions;
- the MLP reshapes the features inside each position.

3.3 Attention from first principles

For one attention head, the input matrix is $X \in \mathbb{R}^{n \times d_{\text{model}}}$. Three learned linear maps produce

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

For causal self-attention, the output at token t is

$$y_t = \sum_{j \le t} \alpha_{tj} v_j, \qquad \alpha_{tj} = \frac{\exp(q_t^\top k_j / \sqrt{d_k})}{\sum_{\ell \le t} \exp(q_t^\top k_\ell / \sqrt{d_k})}. \tag{2}$$

Read this in words.

1. Compare the current query q_t with each earlier key k_j.
2. Convert the scores into nonnegative weights that sum to one.
3. Return the weighted average of the corresponding values.

Worked example 3: a tiny numerical attention head

Take two past keys

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

one current query

$$q = \begin{bmatrix} 2 \\ 0 \end{bmatrix},$$

and values

$$v_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \qquad v_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

The scores are 2 and 0, so the weights are approximately 0.88 and 0.12. The output is therefore

$$y \approx 0.88 \begin{bmatrix} 3 \\ 1 \end{bmatrix} + 0.12 \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 2.64 \\ 1.12 \end{bmatrix}.$$

This is the smallest complete attention computation: compare, normalize, average.

3.4 Why exact attention is expensive

If the sequence length is n, then the score matrix QK^T has size $n \times n$. This is where the quadratic dependence on sequence length comes from. In rough terms:

$$\text{time for the score matrix} = \mathcal{O}(n^2 d_k), \qquad \text{memory for the scores} = \mathcal{O}(n^2).$$

This is the main reason the literature searches for alternatives.

Training cost versus inference cost

During **training**, many implementations process whole sequences in parallel. The quadratic cost appears because every token compares itself with many others. During **autoregressive inference**, the dominant issue is different: the model must keep all past keys and values in a growing KV cache. Exact attention therefore has two efficiency concerns:

- quadratic work over sequence length during training;
- linear cache growth during decoding.

These are related, but they are not the same problem.

3.5 Positional information

Without positional information, attention would only see a set of tokens, not their order. Position can be injected in several ways.

- **Additive positional embeddings:** add a learned or sinusoidal position vector to each token

embedding.

- **RoPE:** rotate query and key coordinates in a position-dependent way so relative position is encoded inside their inner product [15].
- **ALiBi:** bias attention scores by a distance-dependent linear term [16].

For a first reading, the most important point is conceptual:

> position changes the similarity rule itself.

RoPE does this by rotating coordinates; ALiBi does it by adding a distance penalty to the score.

3.6 Training versus inference, and why the KV cache matters

At decoding step t, a Transformer does not want to recompute all old keys and values from scratch. So it stores them. That storage is the KV cache. If there are H heads, sequence length t, and per-head value size d_v, then the cache size grows roughly like

$$\mathcal{O}(tHd_v).$$

For long contexts this becomes a serious deployment bottleneck even if the attention kernel itself is fast.

Worked example 4: why cache compression matters

Suppose one model stores one separate key-value set for each of 32 heads, while another shares the keys and values across groups of 4 query heads. Then the second model stores only 8 key-value sets instead of 32. This is the core idea behind grouped-query attention (GQA): keep many queries, but share some of the stored past. The model still performs exact attention; it simply stores the past more economically.

3.7 MQA, GQA, and MLA in plain language

- **MQA** shares one set of keys and values across all query heads [5]. This minimizes cache size but can be too restrictive.
- **GQA** shares keys and values within groups of heads, which is a compromise between MQA and full multi-head attention [47].
- **MLA** compresses the key-value state into a smaller latent representation before decoding [33, 39]. It is a cache-compression idea rather than a recurrent-state idea.

What to remember from the core ideas

After this section, you should be able to say the following in plain English.

1. A Transformer predicts the next token by building a useful representation of the prefix.
2. Self-attention compares the current query with earlier keys and averages earlier values.

3. Exact attention is expensive because it creates many pairwise query-key interactions.
4. During inference, the KV cache keeps growing with context length.
5. Many later architectures can be understood as changing either the *representation of the past* or the *cost of reading it*.

4 Historical Story in Plain English

Figure 4 gives the historical picture we will keep returning to. The exact derivations are preserved in Appendix A; here the goal is conceptual continuity.

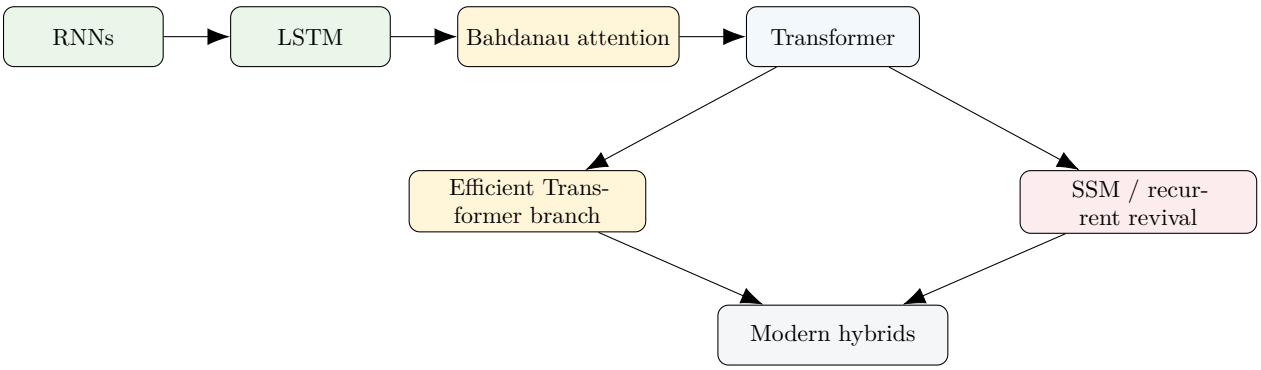

Figure 4: A compact family tree. The field did not move in a straight line; it branched and later recombined.

4.1 RNNs: compress everything into one state

The earliest deep sequence models asked one hidden state to summarize the entire past:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t).$$

The good news is obvious: the memory is fixed-size. The bad news is equally obvious: the whole past is repeatedly compressed into a single vector. Long dependencies are therefore difficult.

Worked example 5: the vanishing-gradient intuition

In a scalar linearized recurrence

$$h_t = a h_{t-1} + b x_t,$$

the influence of h_{t-k} on h_t is proportional to a^k. If $|a| < 1$, then a^k shrinks quickly as k grows. This is the core intuition behind the vanishing-gradient problem: information and gradients can decay exponentially with distance.

4.2 LSTM: keep an additive memory lane

The LSTM changed the recurrence so that a cell state can be copied forward additively:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t.$$

Because the old state is copied rather than remixed from scratch, gradients can propagate more easily.

Worked example 6: why the forget gate helps

Take a scalar cell state with

$$c_{t-1} = 4, \qquad f_t = 0.9, \qquad i_t = 0.1, \qquad g_t = 2.$$

Then

$$c_t = 0.9 \cdot 4 + 0.1 \cdot 2 = 3.8.$$

The old memory mostly survives because the forget gate is near 1. In backpropagation, the derivative of c_t with respect to c_{t-1} is also 0.9, so the gradient can travel through time much more easily than in a recurrence that repeatedly multiplies by unrelated dense matrices.

4.3 Fast weights and Bahdanau attention: retrieve what you need

Fast-weight ideas treated memory as an associative store built from outer products. Bahdanau attention then made the decisive conceptual leap for sequence-to-sequence models: do not compress the whole source sentence into one vector if you can retrieve the relevant source positions directly [3]. This breaks the old encoder bottleneck.

4.4 The Transformer: parallel exact retrieval

The Transformer removes recurrence from the sequence mixer and lets every token directly retrieve information from earlier tokens via attention [4]. This brought two huge benefits:

- strong long-range dependency modeling;
- highly parallel training.

But it also created the modern efficiency problem: exact retrieval is expensive.

Where the detailed historical derivations live

Appendix A contains the derivations for vanilla RNNs, LSTMs, fast weights, Bahdanau attention, and the Transformer. If you are a first-time reader, it is fine to postpone those details until after Sections 5 and 7.

5 Efficient Transformer Ideas, Taught in the Most Intuitive Order

This section can feel confusing because many papers seem to make unrelated tricks. The simplest way to learn it is to ask what each method changes in the core attention formula.

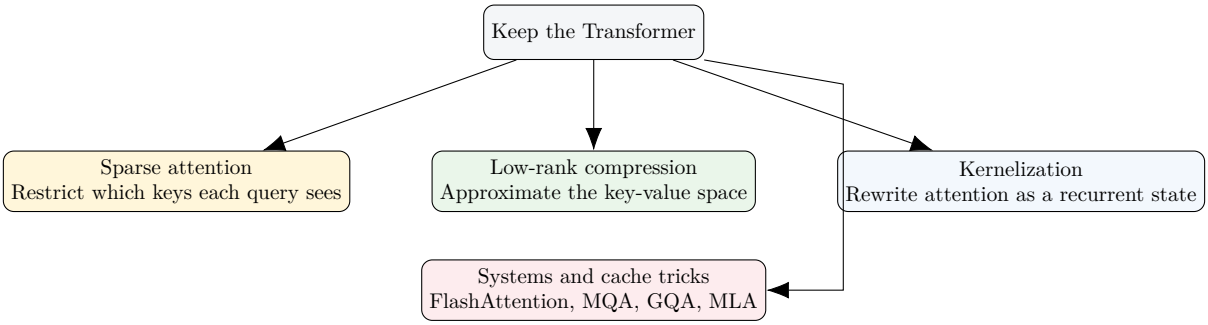

Figure 5: Five ways to make a Transformer more efficient without abandoning the architecture.

5.1 Sparse attention: let each token see only some of the past

Local windows, Longformer, BigBird, and Reformer all reduce cost by limiting which key positions are visible to a query [6–9]. The simplest form is a local window:

$$y_t = \sum_{j=t-w+1}^{t} \alpha_{tj} v_j,$$

where only the last w positions are visible.

Worked example 7: what a local window buys and loses

Suppose a model uses a window size $w = 128$. Token t can inspect the previous 128 tokens exactly, but it cannot directly attend to token $t - 5000$. The gain is clear: the work per token becomes proportional to the window size rather than the full sequence length. The loss is also clear: long-range dependencies are no longer exact unless the pattern includes special global or random connections.

BigBird adds global and random edges so the sparse graph still has useful long-range connectivity [8]. Reformer uses locality-sensitive hashing so similar queries and keys are more likely to land in the same bucket and attend to each other [9]. The full derivations and graph arguments are deferred to Appendix B.

5.2 Low-rank attention: compress the sequence dimension

Linformer and Nyströmformer assume that the full attention pattern can be well approximated in a lower-dimensional subspace [10, 13]. In Linformer, the key and value sequences are projected:

$$K \mapsto EK, \qquad V \mapsto FV,$$

where E and F have many fewer rows than the original sequence length.

Worked example 8: Linformer in one sentence

If a 4096-token sequence can be summarized by 256 learned sequence-direction basis vectors, then attention no longer needs to compare each query with all 4096 original keys. It compares with 256 compressed slots instead. The assumption is that the attention matrix is approximately low-rank.

Nyströmformer makes a related move by selecting or constructing landmark points that approximate the full kernel matrix [13]. These methods are attractive when low-rank structure is present, but they can struggle if the true attention pattern is highly localized or highly irregular.

5.3 Kernelized linear attention: turn the prefix into a finite-state memory

If the softmax kernel is replaced or approximated by an inner product in feature space,

$$\kappa(q, k) \approx \phi(q)^{\mathsf{T}} \phi(k),$$

then the attention numerator becomes

$$\sum_{j \le t} \phi(q_t)^{\mathsf{T}} \phi(k_j) v_j = \phi(q_t)^{\mathsf{T}} \left(\sum_{j \le t} \phi(k_j) v_j^{\mathsf{T}} \right).$$

The term in parentheses is a matrix-valued recurrent state. This is the main bridge from attention to recurrence [11, 14].

Worked example 9: the recurrent state in linear attention

Using the toy keys and values from Section 2, and taking the feature map to be the identity for intuition, define

$$S_2 = k_1 v_1^{\mathsf{T}} + k_2 v_2^{\mathsf{T}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Then

$$S_2^{\mathsf{T}} q = q = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$$

After normalization by the summed feature vector, the readout favors v_1. The important point is not the exact numbers. The important point is that the whole prefix has become one matrix state S_t rather than a growing cache.

Performer adds random-feature approximations that make this idea practical while staying close to the softmax kernel in expectation [14].

5.4 Exact attention can still be made much faster

FlashAttention does not change the attention formula at all [17, 24, 46]. Instead, it changes *how the exact computation is scheduled on hardware*. The key trick is to avoid materializing the full attention matrix in slow memory and instead compute attention in tiles while maintaining an online softmax normalization.

Why FlashAttention matters even in a survey about alternatives

A crucial lesson is: "more efficient" does *not* always mean "use a different architecture." Sometimes the right move is to keep exact attention but implement it in a far more hardware-aware way. This is why the baseline kept improving even as linear and recurrent alternatives became more popular.

5.5 Cache compression: MQA, GQA, and MLA

These methods target inference memory more directly than training complexity. MQA shares the stored key-value state across all query heads [5]. GQA shares it within groups [47]. MLA compresses the key-value representation into a smaller latent form before caching [33, 39].

Where the full derivations live

Appendix B contains the detailed derivations and more technical discussion for FlashAttention, BigBird, Reformer, Linformer, Nyströmformer, linear attention, Performer, and cache-compressed exact attention.

6 Before Modern Beyond-Transformer Models: Why Recurrence and State Spaces Returned

The simplest question here is: if a growing KV cache is expensive, can we store the past in a fixed-size state again, but do it better than old RNNs did?

6.1 A scalar state-space model already explains a lot

Consider the scalar recurrence

$$h_t = ah_{t-1} + bx_t, \qquad y_t = ch_t.$$

If $h_0 = 0$, then repeated substitution gives

$$h_t = bx_t + abx_{t-1} + a^2bx_{t-2} + \cdots .$$

So a recurrence can be unrolled into a convolutional filter. This is a key first-pass fact about state-space models.

Worked example 10: recurrence as a decaying convolution

Take $a = 0.5$ and $b = 1$. Then

$$h_t = x_t + 0.5x_{t-1} + 0.25x_{t-2} + \cdots .$$

The state is simply keeping an exponentially decayed summary of the past. This is already enough to explain why forgetting factors, decays, and time constants appear so often in modern recurrent models.

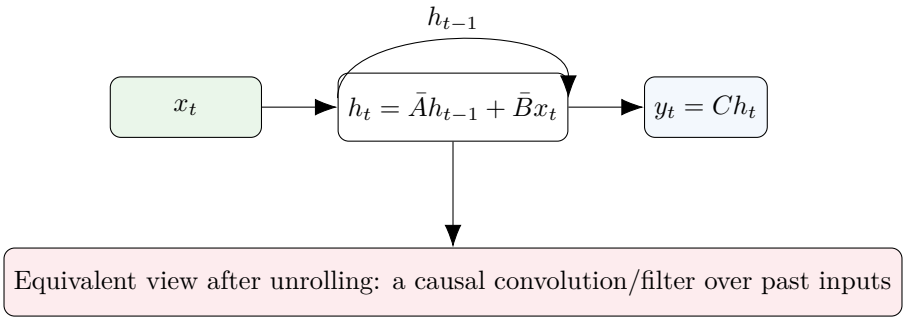

Figure 6: The two canonical views of a linear time-invariant state-space model: recurrence and convolution.

6.2 From continuous time to discrete time

A continuous-time linear state-space model is

$$\dot{h}(t) = Ah(t) + Bx(t), \qquad y(t) = Ch(t).$$

Discretization converts this into a step-by-step recurrence. The core idea is easy to state:

> Assume the input stays roughly constant during a small time interval, solve the differential equation over that interval, and obtain a discrete update.

That is the meaning of phrases such as *zero-order hold*. The full formula is important, but the concept is more important on a first reading.

6.3 HiPPO and S4: summarize the past in a principled basis

HiPPO asks for an online summary of the past signal by maintaining coefficients in a polynomial basis [12]. S4 then chooses special structured state matrices so long-range filters can be computed and trained efficiently [18]. The key interpretation is:

> do not memorize the entire past; instead keep a carefully chosen set of coefficients that summarize it.

6.4 H3, Hyena, and RWKV

H3 showed how to combine SSM-style long filters with multiplicative interactions better suited to language [19]. Hyena made very long implicit convolutions and data-controlled gating practical [20]. RWKV built a recurrent language model with a Transformer-like training interface and a fixed-size inference state [21]. These models all matter historically because they prepared the ground for the 2023–2026 revival of finite-state sequence modeling.

Where the full derivations live

Appendix C gives the full derivations for discrete state-space models, HiPPO, S4, H3, Hyena, and RWKV.

7 The Common Modern Framework: Finite-State Sequence Mixing

This section is the conceptual heart of the survey. If you learn only one technical story from the document, learn this one.

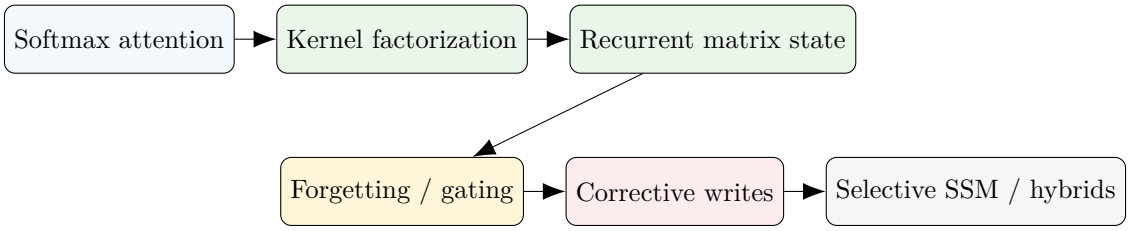

Figure 7: The shortest derivational path that connects many modern models.

7.1 Step 1: start from causal attention

The exact attention readout is

$$y_t = \frac{\sum_{j \le t} \exp(q_t^\top k_j)\, v_j}{\sum_{j \le t} \exp(q_t^\top k_j)}. \tag{3}$$

The computational bottleneck comes from the fact that the current query interacts with every earlier key.

7.2 Step 2: expose a prefix state by kernelization

If

$$\exp(q^\top k) \approx \phi(q)^\top \phi(k),$$

then the numerator becomes

$$\phi(q_t)^\top \left(\sum_{j \le t} \phi(k_j) v_j^\top \right).$$

This motivates the matrix memory

$$S_t = \sum_{j \le t} \phi(k_j) v_j^\top, \qquad z_t = \sum_{j \le t} \phi(k_j), \tag{4}$$

and the readout

$$y_t = \frac{S_t^\top \phi(q_t)}{z_t^\top \phi(q_t)}. \tag{5}$$

Now the whole prefix has been compressed into a finite state.

Worked example 11: linear attention as a state update

Equation (4) immediately implies the recurrence

$$S_t = S_{t-1} + \phi(k_t) v_t^\top.$$

So linear attention is not merely "attention made cheaper." It is also a recurrent memory update. That is why the famous phrase "Transformers are RNNs" captures something real [11].

7.3 Step 3: forgetting yields RetNet-like memory

A fixed decay gives

$$S_t = \lambda S_{t-1} + \phi(k_t) v_t^\top, \qquad 0 \le \lambda \le 1. \tag{6}$$

If $\lambda < 1$, older information gradually decays. This is the simplest path to RetNet-style retention [22].

Worked example 12: what forgetting does

If a memory item is written at time j and there are no later rewrites, then by time t its contribution is multiplied by λ^{t-j}. With $\lambda = 0.95$, a very old item still survives; with $\lambda = 0.5$, it disappears quickly. Forgetting is therefore a learned or chosen time-scale mechanism.

7.4 Step 4: data-dependent forgetting yields GLA-like memory

Replace the fixed scalar decay by a gate:

$$S_t = G_t \odot S_{t-1} + \phi(k_t) v_t^\top. \tag{7}$$

Now the model can decide, based on the current token, how much of the old memory to keep. This is the central idea behind Gated Linear Attention (GLA) [23]. Constant gates reduce back toward RetNet-like behavior.

7.5 Step 5: corrective writes yield DeltaNet-like memory

A different idea is not to append a new value blindly, but to correct the value that the current memory would already predict for the current key. Let

$$\hat{v}_t = S_{t-1}^{\mathsf{T}}\phi(k_t)$$

be the value predicted by the current memory for key k_t. Then the update

$$S_t = S_{t-1} + \eta_t \phi(k_t)\big(v_t - \hat{v}_t\big)^{\mathsf{T}} \tag{8}$$

writes only the residual error. This is the modern delta-rule viewpoint used by DeltaNet [30].

Worked example 13: why the delta rule is different from an additive write

Suppose the memory already predicts a value close to the desired one. Then $v_t - \hat{v}_t$ is small, so the update is small. If the memory predicts badly, the correction is large. In other words, the delta rule is *self-correcting*; it adapts the write size to the current prediction error rather than writing every observation with the same strength.

7.6 Step 6: forgetting plus correction yields Gated DeltaNet

If we combine the previous two ideas, we obtain a family of updates of the form

$$S_t = G_t \odot S_{t-1} + \eta_t \phi(k_t)\big(v_t - \hat{v}_t\big)^{\mathsf{T}}.$$

This combines time-scale control with corrective writing and is the main idea behind Gated DeltaNet [42].

7.7 Step 7: selective state-space models yield Mamba and Mamba-2

A generic discrete state-space model has

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t.$$

In a classical linear time-invariant model, the matrices do not depend on the input. Mamba makes key parts of the update depend on the current token, especially through input-dependent step sizes and parameter selection [25]. This is why people call it a *selective* state-space model. Mamba-2 then re-expresses this family through structured state-space duality (SSD), which clarifies the relation between state-space recurrences and semiseparable matrix operators [36].

7.8 Step 8: matrix memory and gating yield xLSTM

xLSTM revisits the LSTM idea but lets the memory be matrix-valued rather than purely vector-valued [29]. Conceptually this returns to the same theme as linear attention and DeltaNet: use a

richer finite state than a single hidden vector, but preserve the gating intuition that made LSTMs durable.

The shortest possible summary of the common framework

Many modern models differ mainly in *which finite state they keep* and *how they update it*.

Method	Core update idea	Main extra ingredient
Linear attention	additive state	feature-map kernelization
RetNet	additive state + decay	fixed forgetting
GLA	additive state + gate	data-dependent forgetting
DeltaNet	additive state + correction	delta-rule residual write
Gated DeltaNet	decay/gate + correction	two memory controls together
Mamba	state-space recurrence	token-dependent selection
xLSTM	gated recurrent memory	matrix-valued cell state

Where the full derivations live

Appendix D contains the complete derivation chain: softmax attention, kernel factorization, recurrent compression, parallel scan, RetNet, GLA, DeltaNet, Gated DeltaNet, Mamba, Mamba-2 / SSD, and xLSTM.

8 Recent Influential Works (2023–March 2026), Explained Through the Common Framework

It is easy to get lost here because there are many names and each paper uses their own language. The trick is to group them by what they improved.

Four shifts that changed the frontier

From 2023 to March 2026, four shifts were especially visible.

1. **Exact attention became a much stronger baseline** because FlashAttention-2 and FlashAttention-3 made it dramatically more hardware-efficient [24, 46].
2. **Inference cache efficiency moved to the center** through GQA and MLA [33, 39, 47].
3. **Finite-state models became credible large-scale alternatives** through RetNet, Mamba, DeltaNet, xLSTM, and related work [22, 25, 29, 30, 36].
4. **Hybrids became the practical compromise** because exact retrieval and compressed memory solve different problems well [26, 27, 31, 34, 37, 40, 41, 43, 44].

8.1 Exact attention and cache efficiency

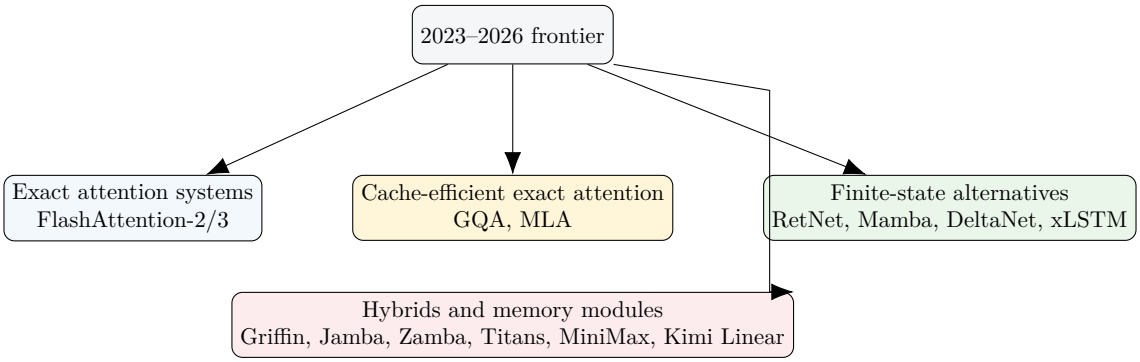

Figure 8: A map of the recent frontier.

Table 2: Recent exact-attention and cache-efficiency milestones.

Work	**Relation to the common framework**	**What it improves**	**What evidence the paper uses**
FlashAttn-2 [24]	Keeps exact softmax attention; changes the algorithmic schedule, not the memory representation.	Better work partitioning and less overhead than earlier exact kernels.	Kernel benchmarks, higher utilization, and end-to-end training/inference speedups.
FlashAttn-3 [46]	Again keeps exact attention; exploits newer Hopper hardware and FP8 pathways.	Higher utilization, overlap of compute and data movement, improved low-precision behavior.	H100 benchmarks, utilization numbers, and numerical-error comparisons.
GQA [47]	Still exact attention; reduces stored past by sharing keys/values across query groups.	Much smaller inference cache with smaller quality loss than full MQA.	Uptraining experiments and quality-versus-speed/cache comparisons.
MLA / DeepSeek-V2,V3 [33, 39]	Still retrieval-based rather than recurrent; compresses the cached key-value state into latent factors.	Large cache savings at long context while retaining strong model quality in large-scale systems.	Large-scale model reports with benchmark results, context-length support, and cache analyses.

The important lesson is that recent progress did not all point away from the Transformer. Some of the strongest practical wins came from making exact attention *cheaper to execute* or *cheaper to cache*.

8.2 Finite-state recurrent and SSM alternatives

Table 3: Recent finite-state alternatives.

Work	Relation to the common framework	Main improvement claimed	How the claim is substantiated
RetNet [22]	Finite-state matrix memory with exponential decay; closely matches Step 3 in Section 7.	Training parallelism together with recurrent inference and decay-controlled memory.	Language-model benchmarks, long-sequence evaluations, and parallel/chunkwise algorithms.
GLA [23]	Adds data-dependent gates to the linear-attention memory.	More expressive memory retention than fixed decays.	Ablations on gating and empirical performance versus other linear-recurrent baselines.
Mamba [25]	Selective state-space model; corresponds to token-dependent state updates in Step 7.	Strong language performance with linear-time selective recurrence and fixed-size inference state.	Across-domain benchmarks, throughput measurements, and long-context evaluations.
Mamba-2 / SSD [36]	Reformulates selective SSMs through structured state-space duality.	Cleaner theory and faster or more flexible algorithms for the same family.	Theoretical derivation, algorithmic speedups, and language-model experiments.
DeltaNet [30]	Implements the delta-rule corrective write of Step 5.	Better use of limited recurrent memory by writing prediction errors rather than raw values.	Synthetic memory tasks, language modeling, and ablations against additive-memory baselines.
Gated DeltaNet [42]	Combines corrective writing with data-dependent forgetting.	Stronger memory control than either pure decay or pure correction alone.	Ablation studies and long-context benchmarks.

Work	Relation to the common framework	Main improvement claimed	How the claim is substantiated
xLSTM [29]	Returns to LSTM-style gating but with richer matrix memory and modern scaling choices.	A modern recurrent architecture with LSTM intuition but much stronger capacity.	Scaling studies, language-model experiments, and comparisons with Transformer/SSM baselines.
HGRN2 [28]	Another recent recurrent line emphasizing expanded hidden-state structure and gating.	Improved efficiency-quality tradeoffs for recurrent language modeling.	Benchmark comparisons and ablations on recurrent design choices.

The common thread is no longer mysterious once you have Section 7 in mind. These papers all ask: how can a fixed-size state be made expressive enough to compete with a growing exact-retrieval cache?

8.3 Hybrids: exact retrieval where it matters, compressed memory elsewhere

Table 4: Recent hybrid models.

Work	Relation to the common framework	Main improvement claimed	How the claim is substantiated
Griffin / Hawk [26]	Mix local attention and linear recurrences (Griffin) or use the recurrent part alone (Hawk).	Better long-context and throughput tradeoffs than full Transformers at comparable scale.	Scaling runs, context extrapolation, and compute-matched comparisons.
Recurrent Gemma [27]	Open models built from Griffin.	Competitive quality with smaller memory footprint and fewer training tokens than Gemma counterparts.	Open-model benchmarks and efficiency comparisons.

Work	Relation to the common framework	Main improvement claimed	How the claim is substantiated
Samba [32]	Interleaves sliding-window attention with Mamba blocks.	Combines local exact retrieval with fixed-state global mixing.	Ablations on layer mixtures and benchmark results.
Jamba / Jamba-1.5 [31, 35]	Interleaves Transformer and Mamba layers, often with MoE.	High throughput and strong long-context quality at large scale.	Large-scale system reports, benchmark suites, and context-window results.
Zamba / Zamba2 [34, 38]	Hybrid Mamba-Transformer language models.	Good quality with fewer attention layers and efficient inference behavior.	Training curves, benchmark comparisons, and architecture ablations.
Hymba [37]	Places attention and SSM-style operations together inside a single layer/head design.	Better small-model efficiency-quality trade-offs and reduced cache pressure.	Small- and medium-scale benchmark suites plus architectural ablations.
Falcon-H1 [44]	Hybrid-head open models mixing attention with SSM-style components.	Strong open-weight quality and efficiency across model sizes with long-context support.	A broad report over several scales, multilingual tasks, and 256K-context evaluations.

The rise of hybrids is a strong empirical hint that no single mixer dominates every workload. Exact attention is very good at pinpoint retrieval. Recurrent state is good for fixed-size memory and cheap decoding. Hybrids try to keep both advantages.

8.4 Explicit memory modules and ultra-long context

Table 5: Recent memory-augmented and ultra-long-context systems.

Work	Relation to the common framework	Main improvement claimed	How the claim is substantiated
Titans [40]	Adds an explicit neural long-term memory alongside attention.	Better use of very long history than either attention-only or pure recurrent baselines.	Needle-in-a-haystack tests, long-context benchmarks, and cross-domain experiments.
MiniMax-01 [41]	Uses lightning attention inside a very large MoE system.	Million-token context scaling with competitive general model quality.	Large-scale report, long-context benchmarks, and throughput claims at large context.
Kimi Linear [43]	Hybrid of Kimi Delta Attention and MLA; extends the Gated DeltaNet line with finer gating.	Reports that a linear-attention hybrid can outperform full attention under fair comparisons while using less cache.	Large pretraining comparisons, long-context decoding throughput, cache-size analysis, and released systems support.

What to conclude from the recent frontier

The frontier did *not* settle on one universal replacement for the Transformer. Instead, the community learned three things.

1. Exact attention remained stronger than many people expected once hardware-aware kernels improved.
2. Fixed-size recurrent memory became much more credible once gating, correction, selection, and matrix memory were developed.
3. Hybrid designs kept winning practical mindshare because they combine exact retrieval with efficient memory compression.

Where the extended recent-paper notes live

Appendix E gives a longer, paper-by-paper discussion of the recent models, including how each one relates to the common framework and what the authors use as evidence for their claims.

9 Comparative Synthesis and Practical Design Rules

Table 6: A comparison table.

Family	What is stored about the past?	Training sequence cost	Inference state	Main strength / main limitation
Exact attention	All past keys and values	Typically quadratic in n	Growing KV cache	Very strong direct retrieval / expensive at long context
Sparse attention	Only selected attention edges	Usually subquadratic	Still a cache, but with structured sparsity	Cheap local access / may miss long dependencies without careful patterns
Low-rank attention	Compressed sequence-direction summaries	Often near-linear in the sequence dimension used internally	Compressed cache or compressed sequence summaries	Good when attention is low-rank / can fail on highly irregular patterns
Linear attention / linear recurrence	Finite matrix or vector state	Linear in n	Fixed-size state	Cheap long decoding / weaker exact retrieval than full attention
Selective SSMs	Finite state with token-dependent dynamics	Linear in n	Fixed-size state	Strong efficiency-quality tradeoff / harder to interpret than plain attention
Hybrids	Some exact retrieval plus some compressed memory	Depends on the mix	Mixed cache + state	Often best practical compromise / more design complexity
Explicit long-term memory	Attention or recurrence plus a separate learned memory module	Depends on the base model	Base state plus external memory	Very long history handling / extra system complexity

9.1 Five design rules that recur across the literature

1. **Ask first whether you need exact retrieval.** If your task needs pinpoint copying from arbitrary earlier positions, pure finite-state models can struggle.
2. **Separate training complexity from inference memory.** A method can train quickly but still have an expensive cache, or the reverse.
3. **Decays, gates, and step sizes are time-scale controls.** Whenever you see them, ask what

old information is being preserved or forgotten.

4. **Corrective updates and selective updates are attempts to make limited memory more expressive.** DeltaNet and Mamba are two different examples of this principle.
5. **Hybrids usually appear when one memory type is not enough.** This is the practical reason they became so common.

What you should now be able to explain in plain English

By this point, a first-time reader should be able to answer all of the following without looking at the equations.

1. Why is exact self-attention expensive?
2. How does linear attention turn a prefix into a fixed-size state?
3. Why does forgetting help recurrent memory?
4. How is a delta-rule write different from simply adding a new outer product?
5. What does "selection" mean in Mamba?
6. Why did hybrid models become so prominent?

If any answer still feels fuzzy, revisit Sections 2, 5, and 7 before going into the appendices.

10 Glossary and Self-Check with Answer Sketches

10.1 Compact glossary

Table 7: A short glossary for a first reading.

Term	Meaning in this survey
Attention	A weighted-sum retrieval mechanism that reads values using query-key similarities.
KV cache	Stored past keys and values used for autoregressive decoding in attention models.
Feature map ϕ	A transformation used to rewrite or approximate a kernel so attention can be expressed as inner products in a new feature space.
Finite-state memory	A recurrent representation of the prefix whose size does not grow with context length.
Decay / forgetting	A multiplicative factor that gradually reduces the influence of old state components.
Delta rule	A corrective update that writes prediction error rather than writing the raw target directly.

Term	Meaning in this survey
Selective SSM	A state-space model whose update depends on the input token, not only on fixed matrices.
Semiseparable matrix	A structured matrix family used to connect state-space recurrences and attention-like operators in Mamba-2 / SSD.
Hybrid model	A model that combines more than one memory style, such as attention plus recurrence.
Long-context model	Any sequence model designed to remain accurate and efficient on very long prefixes.

10.2 Self-check questions with short answer sketches

1. **Why is attention a weighted sum?**
   Answer sketch. Because each output token is computed as $y_t = \sum_j \alpha_{tj} v_j$, where the coefficients α_{tj} are normalized query-key similarity scores.

2. **Why does exact self-attention become quadratic in sequence length?**
   Answer sketch. Each of the n queries typically interacts with about n keys, producing about n^2 pairwise scores.

3. **What is the conceptual difference between a KV cache and a recurrent state?**
   Answer sketch. A KV cache stores many past token-specific items explicitly and therefore grows with context; a recurrent state stores a compressed summary of the past in fixed size.

4. **What is kernelized linear attention in plain language?**
   Answer sketch. It rewrites the attention weights so the whole prefix can be summarized by a matrix state such as $S_t = \sum_{j \le t} \phi(k_j) v_j^\top$.

5. **Why does forgetting matter?**
   Answer sketch. Because a fixed-size memory cannot preserve everything equally forever; decays and gates let the model choose time scales and suppress stale information.

6. **Why is the delta rule not just another additive write?**
   Answer sketch. Because it writes only the difference between the desired value and the value already predicted by the current memory.

7. **What is Mamba's selectivity in plain language?**
   Answer sketch. The token changes how the state is updated, so the model can retain different information for different inputs rather than using one fixed transition everywhere.

8. **Why did the community not simply replace Transformers with one recurrent model?**
   Answer sketch. Because exact retrieval remains very strong, especially once FlashAttention and cache optimizations improved it; hybrids often give a better tradeoff than any single mixer family.

How to use the appendices

The appendices preserve the detailed derivations and paper-by-paper technical notes. They are not required on a first pass. Their role is to make the document usable on a first pass while remaining historically complete.

A Historical Roots: From RNNs to the Transformer

Learning goals. After this section, you should understand the historical problems that motivated attention, the Transformer, and later efficient or beyond-Transformer variants.

A.1 A short timeline from roots to leaves

Table 8 gives the high-level map before we dive into details.

Table 8: A compact historical timeline from roots to the recent frontier.

Year	Work / family	Why it mattered
1992	History compression / fast memory ideas	Early statement of the sequence-compression problem and the idea that a recurrent state can summarize long histories.
1997	LSTM	Introduced gated additive memory and the constant-error path for long-range gradients.
2014–2015	Bahdanau / Luong attention	Replaced a single fixed encoder summary with a learned weighted sum over encoder states.
2017	Transformer	Made self-attention the dominant sequence mixer by enabling full parallel training and direct token-to-token access.
2019	Transformer-XL / MQA	Exposed the cache problem and the importance of efficient autoregressive decoding.
2019–2020	Sparse Transformer / Longformer / BigBird / Reformer	Early major attempts to reduce the quadratic cost of attention without abandoning the Transformer template.
2020–2021	Linformer / Performer / Nyströmformer	Low-rank and kernelized approximations of attention.
2022	FlashAttention	Showed that exact attention can be made far more practical through I/O-aware kernels.
2020–2022	HiPPO / S4	Put state-space models on firm mathematical ground and made them competitive for long-sequence modeling.

Year	Work / family	Why it mattered
2022–2023	H3 / Hyena / RWKV	Re-opened the search for strong attention-free or mostly attention-free language models.
2023	RetNet / GLA / Mamba	Recast long-context modeling through recurrent state, gating, and selective state spaces.
2024	Mamba-2 / DeltaNet / xLSTM / Griffin	Unified state-space and attention views, improved memory correction, revisited LSTMs, and popularized hybrids.
2024–2025	RecurrentGemma / Jamba / Zamba / Hymba / DeepSeek MLA	Showed that open models and production-minded systems increasingly mix attention with recurrent or compressed-memory components.
2025	Titans / MiniMax-01 / Kimi Linear / Falcon-H1	Pushed toward explicit long-term memory, million-token context, stronger linear/hybrid attention, and hybrid-head families.

A.2 Vanilla RNNs and the vanishing-gradient problem

A simple recurrent neural network updates

$$h_t = \phi(W x_t + U h_{t-1} + b). \tag{9}$$

The problem is not that Eq. (9) cannot, in principle, store the past. The problem is that learning long-term dependencies through gradient descent is difficult.

Let $D_t = \operatorname{diag}(\phi'(W x_t + U h_{t-1} + b))$. Then

$$\frac{\partial h_t}{\partial h_{t-1}} = D_t U. \tag{10}$$

So for $T > t$,

$$\frac{\partial h_T}{\partial h_t} = \prod_{i=t+1}^{T} D_i U. \tag{11}$$

Taking norms,

$$\left\| \frac{\partial h_T}{\partial h_t} \right\| \leq \prod_{i=t+1}^{T} \|D_i\| \, \|U\| \,. \tag{12}$$

If the typical factor is less than 1, the gradient vanishes. If it is greater than 1, the gradient explodes. This is the classic long-range training problem.

Historical importance. Understanding Eq. (12) is essential because most later sequence-model inventions can be read as answers to one of two questions:

- How do we create better paths for information and gradients through time?
- How do we let the model access earlier tokens without forcing everything through one fragile vector state?

A.3 LSTM: deriving gated additive memory from the gradient problem

The Long Short-Term Memory (LSTM) cell answers the first question. Instead of writing

$$h_t = \phi(W x_t + U h_{t-1}),$$

it introduces a cell state c_t with an additive update:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \tag{13}$$
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \tag{14}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \tag{15}$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \tag{16}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{17}$$
$$h_t = o_t \odot \tanh(c_t). \tag{18}$$

Why this helps. Differentiate the cell update with respect to the previous cell:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t. \tag{19}$$

Across many steps,

$$\frac{\partial c_T}{\partial c_t} = \prod_{i=t+1}^{T} f_i. \tag{20}$$

Because each forget gate coordinate can be close to 1, the model can preserve information and gradients along selected coordinates much better than a vanilla RNN.

Why the LSTM is historically important. The LSTM introduced three ideas that keep returning in modern models:

- **additive memory updates,**
- **gating for selective write / keep / expose,**
- **a separation between internal memory and exposed hidden state.**

Modern models such as xLSTM and some linear-recurrent hybrids can be read as large-scale descendants of these ideas.

A.4 Fast weights and associative memory

A different historical branch tried to make the recurrent state more expressive by turning it into a matrix. A simple fast-weight rule is

$$W_t = \lambda W_{t-1} + v_t k_t^\mathsf{T}, \qquad y_t = W_t q_t. \tag{21}$$

This is already remarkably close to several modern models. Unrolling,

$$W_t = \sum_{j=1}^{t} \lambda^{t-j} v_j k_j^\mathsf{T}, \tag{22}$$

and therefore

$$y_t = \sum_{j=1}^{t} \lambda^{t-j} v_j (k_j^\mathsf{T} q_t). \tag{23}$$

That is an attention-like weighted sum over the past, but implemented through a recurrent matrix state instead of an explicit attention matrix.

Why this matters historically. Many current "linear attention" and "matrix memory" architectures are better understood as refined, trainable, hardware-aware descendants of Eq. (21).

A.5 Bahdanau attention: solving the seq2seq bottleneck

Early encoder–decoder models compressed the full input sequence into one fixed vector. That created a bottleneck. Bahdanau attention removes the bottleneck by letting the decoder compute a fresh weighted sum of encoder states at each decoding step:

$$c_t = \sum_{j=1}^{n} \alpha_{tj} h_j^{\text{enc}}, \qquad \alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{i=1}^{n} \exp(e_{ti})}, \tag{24}$$

with an energy function such as

$$e_{tj} = a(s_{t-1}^{\text{dec}}, h_j^{\text{enc}}). \tag{25}$$

Derivation idea. The fixed vector is replaced by a convex combination of encoder states. So instead of forcing the decoder to store the whole source sentence in one vector, the decoder learns *where to look* in the source sequence at each step.

Why this matters historically. Bahdanau attention is the conceptual bridge from recurrent sequence models to the Transformer. It established that direct content-based access to earlier hidden states is a powerful alternative to compressing everything into one recurrent state.

A.6 The Transformer: deriving scaled dot-product attention and parallel sequence mixing

The Transformer removes recurrence from the main token-mixing path and uses self-attention as the core mixer [4]. For one head:

$$Y = \mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right)V. \tag{26}$$

With multiple heads:

$$\mathrm{MHA}(X) = \mathrm{Concat}(Y^{(1)}, \ldots, Y^{(H)})W_O.$$

Problem formulation. RNNs process tokens sequentially. That makes training hard to parallelize over time. The Transformer asks: can we let every token directly access earlier tokens while still training all positions in parallel?

Derivation. For each query token, compute similarity scores against all keys, normalize with a softmax, then average the values using those weights. Because all pairwise scores for a sequence can be formed by the matrix product QK^{T}, the training computation parallelizes across positions.

Why the Transformer won. The Transformer provided:

- direct token-to-token communication,
- shorter gradient paths than RNNs,
- highly parallel training,
- strong scaling behavior with data and model size.

Its main weakness is the quadratic cost of dense attention and the growing KV cache during decoding. The next sections survey the many attempts to fix those weaknesses.

A.7 A short historical summary

Historically, the field alternated between two design philosophies:

- **compression:** summarize the past in a recurrent state;
- **explicit retrieval:** keep or reconstruct enough past token information to retrieve what is needed later.

RNNs and LSTMs emphasized compression. Bahdanau attention and the Transformer emphasized explicit retrieval. The modern literature is largely about *combining* those two philosophies.

B Efficient Transformer Architectures

Learning goals. After this section, you should understand the main ways of making Transformers more efficient *without* fully abandoning the Transformer template: exact-attention systems improvements, sparsity, low-rank structure, kernelization, and cache compression.

B.1 Exact attention can still be optimized: FlashAttention

Before replacing attention, it is crucial to understand how far exact attention can be pushed.

B.1.1 The problem

Naïvely computing attention forms the full score matrix $S = QK^\top/\sqrt{d_k}$, materializes the softmax probabilities, and then multiplies by V. This moves large tensors between high-bandwidth memory and on-chip memory. The main cost is often *memory traffic*, not floating-point arithmetic.

B.1.2 The key derivation: online softmax in tiles

Consider one query row with scores $s_1, \dots, s_n$. A numerically stable softmax uses the row maximum $m = \max_j s_j$:

$$\sum_j e^{s_j} = e^m \sum_j e^{s_j - m}.$$

Suppose the keys and values are processed in blocks. If after some blocks we have stored

$$m^{\text{old}} = \max(\text{scores seen so far}), \qquad \ell^{\text{old}} = \sum_{\text{old}} e^{s_j - m^{\text{old}}},$$

and an output accumulator

$$o^{\text{old}} = \sum_{\text{old}} e^{s_j - m^{\text{old}}} v_j,$$

then when a new block arrives with block maximum m^{blk}, the new row maximum is

$$m^{\text{new}} = \max(m^{\text{old}}, m^{\text{blk}}). \tag{27}$$

The denominator updates as

$$\ell^{\text{new}} = e^{m^{\text{old}} - m^{\text{new}}} \ell^{\text{old}} + \sum_{\text{blk}} e^{s_j - m^{\text{new}}}. \tag{28}$$

The numerator accumulator updates as

$$o^{\text{new}} = e^{m^{\text{old}} - m^{\text{new}}} o^{\text{old}} + \sum_{\text{blk}} e^{s_j - m^{\text{new}}} v_j. \tag{29}$$

At the end,

$$y = \frac{o^{\text{final}}}{\ell^{\text{final}}}. \tag{30}$$

Why this matters. Equations (28)–(30) prove that *exact* softmax attention can be computed block by block without materializing the full attention matrix.

Algorithm 1 FlashAttention idea for one block of query rows

1: Initialize row maxima $m_i \leftarrow -\infty$, row denominators $\ell_i \leftarrow 0$, outputs $o_i \leftarrow 0$
2: **for** each key/value tile $(K^{(b)}, V^{(b)})$ **do**
3: Compute score tile $S^{(b)} = Q(K^{(b)})^\top / \sqrt{d_k}$
4: Compute blockwise maxima $m_i^{(b)} = \max_j S_{ij}^{(b)}$
5: Update $m_i \leftarrow \max(m_i, m_i^{(b)})$
6: Rescale old ℓ_i, o_i and add block contributions using Eqs. (28)–(29)
7: **end for**
8: Return $y_i = o_i / \ell_i$

B.1.3 FlashAttention-2 and FlashAttention-3

FlashAttention-2 keeps the same online-softmax derivation but improves GPU work partitioning and reduces non-matmul overhead [24]. FlashAttention-3 keeps exact attention again but exploits Hopper-specific asynchrony and low-precision support to reach much higher hardware utilization [46]. These works are historically important because they continually strengthen the exact-attention baseline that all alternatives must beat.

B.2 Sparse attention

Sparse attention reduces cost by computing only a subset of pairwise token interactions.

B.2.1 Local-window attention

The simplest sparse rule lets token t attend only to the most recent w tokens:

$$y_t = \sum_{j=\max(1,t-w+1)}^{t} \alpha_{tj} v_j. \tag{31}$$

The cost becomes $\mathcal{O}(nw)$ instead of $\mathcal{O}(n^2)$.

What is gained and lost? Local windows are cheap and work well when most useful information is nearby. But one layer cannot directly connect widely separated tokens.

B.2.2 Longformer and the idea of local plus special tokens

Longformer combines local windows with selected global tokens [7]. A global token can attend to everyone and everyone can attend to it. This produces an efficient graph with short paths for important positions such as classification tokens or question tokens.

B.2.3 BigBird: local + random + global

BigBird uses three edge types [8]:

- local window edges,
- a small number of random edges,
- a few global tokens.

The cost is linear in n up to the chosen sparsity constants.

Why BigBird mattered theoretically. Theorem statements in the BigBird paper show that with suitable sparse patterns, sparse Transformers retain strong expressivity properties such as universal approximation and Turing completeness. A full proof is outside the scope of this tutorial, but the intuition is easy:

- global tokens provide hubs that quickly connect distant parts of the sequence;
- local edges preserve short-range structure;
- random edges improve connectivity and reduce graph diameter.

A simple path-length intuition. If every token can reach a global token in one hop, and a global token can reach every other token in one more hop, then long-range communication can happen in very few layers. This is why sparse patterns with global nodes can preserve much of the power of dense attention.

B.2.4 Reformer: approximate nearest-neighbor attention via LSH

Reformer uses locality-sensitive hashing (LSH) to place similar queries/keys into the same buckets [9]. Attention is then computed only within each bucket.

Derivation idea. Approximate nearest-neighbor search replaces full pairwise comparison. If similar vectors collide under the hash with high probability, then sorting tokens by bucket places likely neighbors near each other. A simple LSH-style rule uses a random projection matrix R and hashes according to the sign or argmax pattern after projection.

Why this helps. Full attention asks every query to compare with all keys. Reformer first *partitions* the keys so each query only compares within a small candidate set.

What is the tradeoff? The speedup comes from avoiding many comparisons, but performance depends on whether the hashing rule preserves the truly important interactions.

B.3 Low-rank attention

Low-rank methods assume the attention map or its action on values can be represented with a low-dimensional bottleneck.

B.3.1 Linformer

Let

$$P = \mathrm{softmax}\left(\frac{QK^\mathsf{T}}{\sqrt{d_k}} + M\right) \in \mathbb{R}^{n\times n}.$$

Linformer argues that P is approximately low-rank [10]. If $P \approx \tilde{P}$ with rank $r \ll n$, then PV can be approximated through a smaller latent dimension.

Instead of explicitly computing a truncated singular value decomposition of P, Linformer learns sequence projections:

$$\tilde{K} = E_K K, \qquad \tilde{V} = E_V V, \qquad E_K, E_V \in \mathbb{R}^{k\times n}, \tag{32}$$

and then computes

$$Y \approx \mathrm{softmax}\left(\frac{Q\tilde{K}^\mathsf{T}}{\sqrt{d_k}}\right)\tilde{V}. \tag{33}$$

If $k \ll n$, the complexity becomes linear in n up to the fixed bottleneck size k.

Intuition. Linformer says: the sequence dimension of the keys and values may be compressible, so project it before computing attention.

B.3.2 Nyströmformer

Nyströmformer chooses m landmark points and approximates the full attention matrix by a Nyström factorization [13]. A simplified form is

$$A(Q,K) \approx A(Q,\tilde{K})\, A(\tilde{Q},\tilde{K})^\dagger\, A(\tilde{Q},K), \tag{34}$$

where $\tilde{Q}, \tilde{K}$ are landmark queries and keys. The basic idea is again low-rank structure, but expressed through landmark interpolation rather than learned linear projections.

B.4 Kernelized linear attention

Kernelized attention replaces the softmax kernel by an inner product in a feature space.

B.4.1 Linear attention from kernel factorization

Suppose we can find a feature map $\phi : \mathbb{R}^{d_k} \to \mathbb{R}^m$ such that

$$\exp(q^\mathsf{T} k/\sqrt{d_k}) \approx \phi(q)^\mathsf{T}\phi(k). \tag{35}$$

Then causal attention becomes

$$y_t \approx \frac{\sum_{j\le t} \phi(q_t)^\mathsf{T}\phi(k_j)v_j}{\sum_{j\le t} \phi(q_t)^\mathsf{T}\phi(k_j)} \tag{36}$$

$$= \frac{\phi(q_t)^\mathsf{T}\left(\sum_{j\le t} \phi(k_j)v_j^\mathsf{T}\right)}{\phi(q_t)^\mathsf{T}\left(\sum_{j\le t} \phi(k_j)\right)}. \tag{37}$$

Define

$$S_t = \sum_{j\le t} \phi(k_j)v_j^\mathsf{T}, \qquad z_t = \sum_{j\le t} \phi(k_j). \tag{38}$$

Then

$$y_t \approx \frac{\phi(q_t)^\mathsf{T} S_t}{\phi(q_t)^\mathsf{T} z_t}, \qquad S_t = S_{t-1} + \phi(k_t)v_t^\mathsf{T}, \quad z_t = z_{t-1} + \phi(k_t). \tag{39}$$

This is the core derivation behind the famous statement that *Transformers are RNNs* in the linear-attention setting [11].

B.4.2 Performer and FAVOR+

Performer makes the kernelization more principled through positive random features [14]. The goal is to approximate the softmax kernel with low-variance random features while preserving numerical stability.

A helpful identity is

$$e^{q^\mathsf{T}k} = e^{\|q\|^2/2}\, e^{\|k\|^2/2}\, e^{-\|q-k\|^2/2}. \tag{40}$$

The last factor is Gaussian-like and admits random-feature approximations. FAVOR+ chooses a positive random feature map ϕ so that

$$\mathbb{E}[\phi(q)^\mathsf{T}\phi(k)] = e^{q^\mathsf{T}k}.$$

Plugging this into Eq. (39) yields a linear-time approximation to softmax attention.

Why Performer mattered. Performer turned the general kernel trick into a concrete and widely used approximation scheme, making "linear attention" a major family rather than a vague idea.

B.5 Exact attention but smaller cache: MQA, GQA, and MLA

Section 2 already introduced the cache formulas. Here we explain why these features matter historically.

MQA. Multi-query attention shares keys and values across heads, shrinking the autoregressive cache dramatically [5].

GQA. Grouped-query attention generalizes MQA by using a moderate number of KV groups [47]. It became extremely influential in practical decoder-only language models because it preserves quality better than pure MQA while still giving most of the decoding speed gain.

MLA. Multi-head latent attention compresses key-value information into a smaller latent state and reconstructs head-specific keys and values on the fly [33, 39]. MLA is historically important because it changed the practical systems conversation: many modern efficient-attention discussions are no longer only about *quadratic training complexity*; they are about *how expensive the inference cache is per token.*

B.6 What to remember from efficient Transformers

Efficient Transformer methods usually change *one* of the following:

- **the attention pattern** (sparse attention),
- **the dimensionality** (low-rank attention),
- **the kernel** (linear attention),
- **the implementation** (FlashAttention),
- **the stored inference state** (MQA, GQA, MLA).

They still inherit the Transformer worldview: attention remains central.

C Architectures Beyond the Transformer Before the Modern 2023 Frontier

Learning goals. After this section, you should understand the pre-2023 lines that made recurrent, convolutional, and state-space alternatives credible.

C.1 State-space models: from continuous time to discrete recurrence

A continuous-time linear state-space model is

$$\dot{h}(t) = Ah(t) + Bu(t), \qquad y(t) = Ch(t) + Du(t). \tag{41}$$

If the input is held constant over a time step of length Δ (zero-order hold), discretization gives

$$h_t = \bar{A}h_{t-1} + \bar{B}u_t, \qquad \bar{A} = e^{\Delta A}, \qquad \bar{B} = \int_0^{\Delta} e^{(\Delta-\tau)A} B \, d\tau. \tag{42}$$

If A is invertible,

$$\bar{B} = A^{-1}(e^{\Delta A} - I)B.$$

Unrolling the recurrence. Repeated substitution gives

$$h_t = \sum_{j=1}^{t} \bar{A}^{\,t-j} \bar{B}\, u_j, \qquad y_t = \sum_{j=1}^{t} C\bar{A}^{\,t-j} \bar{B}\, u_j + Du_t. \tag{43}$$

So a linear time-invariant SSM is also a convolution with kernel

$$K_\ell = C\bar{A}^{\ell}\bar{B}. \tag{44}$$

This is the mathematical bridge between recurrence and convolution.

C.2 HiPPO: a principled online compression of the past

HiPPO asks: if we want a fixed-size state to store the history of a continuous signal, what is the *best* state in a principled approximation sense [12]?

Problem formulation. At each time t, let the past signal on $[0, t]$ be approximated by the first N basis functions of an orthogonal polynomial family. The coefficients of that best approximation form the state $c(t) \in \mathbb{R}^N$.

Key derivation idea. Because the interval $[0, t]$ expands with time, the basis itself changes with t. Differentiating the optimal projection coefficients with respect to time yields a linear ODE

$$\dot{c}(t) = \frac{1}{t} Ac(t) + \frac{1}{t} Bf(t), \tag{45}$$

where A and B depend on the choice of orthogonal basis.

For the scaled Legendre basis (HiPPO-LegS), the entries are

$$A_{nk} = -\begin{cases} \sqrt{(2n+1)(2k+1)}, & n > k, \\ n+1, & n = k, \\ 0, & n < k, \end{cases} \qquad B_n = \sqrt{2n+1}. \tag{46}$$

This matrix is the famous HiPPO matrix.

Why this matters. HiPPO says that sequence compression can be *mathematically derived*, not only heuristically designed. That insight is the root of the later S4 family.

C.3 S4: making state-space models practical and trainable

HiPPO gave a beautiful memory mechanism, but not yet a practical large-scale layer. S4 (*Structured State Spaces*) makes that line computationally viable [18].

Problem formulation. Directly computing the kernel $K_\ell = C\bar{A}^\ell \bar{B}$ for many steps is expensive if $\bar{A}$ is a dense matrix. S4 looks for a parameterization of A that preserves expressive power but enables fast kernel computation.

Key derivation. S4 writes the continuous-time state matrix as a diagonal-plus-low-rank (DPLR) form after a change of basis:

$$A = \Lambda + PQ^\mathsf{T}, \tag{47}$$

where Λ is diagonal (or normal) in the transformed basis. Then powers or resolvents of A can be reduced to structured linear-algebra operations related to the Cauchy kernel. In practical terms, S4 computes the convolution kernel in the frequency domain and uses FFT-based convolution for sequence processing.

Why S4 was historically important. It made state-space models competitive on long-range benchmarks and changed the conversation from "SSMs are elegant but weak" to "SSMs are serious long-sequence backbones."

C.4 H3: why early SSMs struggled on language

H3 (*Hungry Hungry Hippos*) asked a critical question: why were SSMs strong on some sequential tasks but weaker on language [19]? The paper identified two deficits:

- **recall:** retrieving specific earlier tokens;
- **comparison:** comparing current features with earlier features.

H3 adds multiplicative interactions and a shift-like discrete memory component to improve language modeling. Historically, H3 mattered because it diagnosed language-specific weaknesses that directly motivated later models such as Hyena and Mamba.

C.5 Hyena: long convolutions with data-controlled gating

Hyena proposes an attention-free operator based on long convolutions and multiplicative gating [20]. A simplified Hyena layer is

$$u^{(0)} = W_0 x, \tag{48}$$

$$u^{(\ell)} = (h_\ell * u^{(\ell-1)}) \odot (W_\ell x), \qquad \ell = 1, \dots, L, \tag{49}$$

$$y = W_{\text{out}} u^{(L)}. \tag{50}$$

Derivation idea. A single long convolution is linear and might be too weak. Hyena repeatedly interleaves convolution with data-dependent gating. So the operator becomes nonlinear and content-sensitive without ever forming pairwise attention scores.

Why Hyena mattered. It showed that a purely attention-free architecture could be competitive on language at moderate scale and extremely fast at long sequence lengths. That reopened the long-convolution branch as a serious alternative.

C.6 RWKV: a bridge from linear attention to recurrent language models

RWKV combines linear-attention-style time mixing with recurrent inference [21]. The exact formulas evolve across versions, but the key idea is that the model can be written both in a parallel training form and in a recurrent inference form. Historically, RWKV matters because it popularized the idea that an RNN-like architecture could again scale to language-model sizes while keeping constant-state decoding.

C.7 Why these roots still matter today

These pre-2023 beyond-Transformer lines contributed four reusable ideas:

- **principled compression** (HiPPO),
- **practical structured state spaces** (S4),
- **diagnosis of language-specific failure modes** (H3),
- **attention-free but expressive operators** (Hyena, RWKV).

The 2023–2026 frontier did not appear from nowhere; it built directly on these ideas.

D The Common Modern Framework: Finite-State Sequence Mixing

Learning goals. After this section, you should be able to derive the main modern recurrent/SSM families from one common framework and understand why this framework has become the conceptual center of the field.

D.1 Why this framework is the centerpiece

The modern literature contains many names — linear attention, retention, gated recurrence, delta-rule memory, selective SSMs, matrix LSTMs — but the most reusable abstraction is:

> **Finite-state sequence mixing:** compress the prefix into a fixed-size state, update that state with additive writes, forgetting, correction, or input-dependent dynamics, and recover parallel training through associativity, scan, or chunking.

This framework directly explains linear attention, RetNet, Gated Linear Attention, DeltaNet, Gated DeltaNet, Mamba, Mamba-2, xLSTM, and several hybrids.

D.2 Step 1: start from causal softmax attention

At time t,

$$y_t = \frac{\sum_{j\le t} \exp(q_t^\top k_j/\sqrt{d_k})\, v_j}{\sum_{j\le t} \exp(q_t^\top k_j/\sqrt{d_k})}. \tag{51}$$

The numerator is a similarity-weighted sum of values; the denominator normalizes the weights.

D.3 Step 2: replace softmax by a feature-space inner product

Assume

$$\exp(q^\top k/\sqrt{d_k}) \approx \phi(q)^\top \phi(k) \tag{52}$$

for some feature map ϕ. Then

$$y_t \approx \frac{\sum_{j\le t} \phi(q_t)^\top \phi(k_j) v_j}{\sum_{j\le t} \phi(q_t)^\top \phi(k_j)} \tag{53}$$

$$= \frac{\phi(q_t)^\top \left(\sum_{j\le t} \phi(k_j) v_j^\top\right)}{\phi(q_t)^\top \left(\sum_{j\le t} \phi(k_j)\right)}. \tag{54}$$

Define

$$S_t := \sum_{j\le t} \phi(k_j) v_j^\top, \qquad z_t := \sum_{j\le t} \phi(k_j). \tag{55}$$

Then

$$y_t \approx \frac{\phi(q_t)^\top S_t}{\phi(q_t)^\top z_t}. \tag{56}$$

D.4 Step 3: expose the recurrent state

From Eq. (55),

$$S_t = S_{t-1} + \phi(k_t) v_t^\top, \qquad z_t = z_{t-1} + \phi(k_t). \tag{57}$$

The whole prefix is now summarized by the fixed-size pair (S_t, z_t) rather than by all past keys and values. This is the basic linear-attention recurrence.

Example 1 (Why matrix memory appears naturally)**.** *If $\phi(k_t) \in \mathbb{R}^m$ and $v_t \in \mathbb{R}^{d_v}$, then the outer product $\phi(k_t)v_t^\top$ is an $m \times d_v$ matrix. That is why a matrix-valued recurrent state appears naturally in linear attention and fast-weight memory models.*

D.5 Step 4: why associativity recovers parallel training

A generic affine recurrence is

$$s_t = A_t s_{t-1} + b_t. \tag{58}$$

Represent one time step by the affine map $\mathcal{T}_t(s) = A_t s + b_t$. For two consecutive steps,

$$\mathcal{T}_2(\mathcal{T}_1(s)) = A_2 A_1 s + A_2 b_1 + b_2. \tag{59}$$

So the composition law is

$$(A_2, b_2) \circ (A_1, b_1) = (A_2 A_1,\ A_2 b_1 + b_2). \tag{60}$$

Because this rule is associative, segment summaries can be combined in parallel by prefix-scan algorithms. This is the algebraic reason recurrent models can still have parallel training.

D.6 Step 5: fixed forgetting gives RetNet

A simple improvement over Eq. (57) is to add exponential decay:

$$S_t = \lambda S_{t-1} + k_t v_t^\top, \qquad 0 < \lambda \leq 1. \tag{61}$$

Unrolling:

$$S_t = \sum_{j=1}^{t} \lambda^{t-j} k_j v_j^\top. \tag{62}$$

Reading with the query gives

$$y_t = q_t^\top S_t = \sum_{j \leq t} \lambda^{t-j} (q_t^\top k_j) v_j. \tag{63}$$

This is the core retention derivation of RetNet [22].

Interpretation. RetNet is an attention-like kernel with explicit recency bias. The same operator has three useful views:

- a **parallel** lower-triangular kernel for training,
- a **recurrent** state update for inference,
- a **chunkwise** view that mixes within chunks and summarizes across chunks.

D.7 Step 6: data-dependent forgetting gives Gated Linear Attention

Replace the fixed scalar decay by a gate G_t:

$$S_t = G_t \odot S_{t-1} + k_t v_t^\top. \tag{64}$$

Unrolling,

$$S_t = \sum_{j=1}^{t} \left(\prod_{i=j+1}^{t} G_i \right) \odot (k_j v_j^\top). \tag{65}$$

Therefore

$$y_t = q_t^\top S_t. \tag{66}$$

Compared with RetNet, the effective decay is now content dependent. The model can forget different parts of the state at different rates.

Why this matters. Data-dependent forgetting is one of the most influential ideas of the modern line. It lets the model decide *what to keep* and *what to erase* rather than using a fixed recency schedule.

D.8 Step 7: delta-rule correction gives DeltaNet

Additive writes can interfere with older memory. DeltaNet instead treats the memory as an online linear regressor that should map a key to its value.

Problem formulation. Given current memory S, key k_t, and value v_t, minimize the one-step loss

$$\mathcal{L}_t(S) = \frac{1}{2} \|Sk_t - v_t\|_2^2 . \tag{67}$$

Its gradient is

$$\nabla_S \mathcal{L}_t(S) = (Sk_t - v_t)k_t^\top. \tag{68}$$

A gradient step with rate η_t gives

$$S_t = S_{t-1} - \eta_t (S_{t-1}k_t - v_t)k_t^\top \tag{69}$$

$$= S_{t-1} + \eta_t (v_t - r_t)k_t^\top, \qquad r_t := S_{t-1}k_t. \tag{70}$$

Interpretation. Retrieve the current prediction r_t, compute the residual error $v_t - r_t$, and write only the correction. This is the key DeltaNet idea [30].

A useful rearrangement is

$$S_t = S_{t-1}(I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top. \tag{71}$$

The factor $(I - \eta_t k_t k_t^\top)$ shows that the memory is partially erased in the current key direction before the new association is written.

D.9 Step 8: vectorization makes DeltaNet parallelizable

Use the identity

$$\mathrm{vec}(AXB) = (B^\top \otimes A)\,\mathrm{vec}(X).$$

Vectorizing Eq. (71) gives

$$\mathrm{vec}(S_t) = \underbrace{\left((I - \eta_t k_t k_t^\mathsf{T})^\mathsf{T} \otimes I\right)}_{=:\tilde{A}_t} \mathrm{vec}(S_{t-1}) + \underbrace{\eta_t (k_t \otimes I) v_t}_{=:\tilde{b}_t}. \tag{72}$$

So DeltaNet also fits the affine scan form in Eq. (58). This is the bridge from the delta rule to hardware-efficient parallel training.

D.10 Step 9: combining forgetting and correction gives Gated DeltaNet

Gated DeltaNet simply performs both operations:

$$\bar{S}_{t-1} = G_t \odot S_{t-1}, \tag{73}$$

$$r_t = \bar{S}_{t-1} k_t, \tag{74}$$

$$S_t = \bar{S}_{t-1} + \eta_t (v_t - r_t) k_t^\mathsf{T}. \tag{75}$$

This can be read as

$$\text{forget} \to \text{retrieve} \to \text{correct} \to \text{write}.$$

That short slogan is one of the most useful ways to remember the model.

D.11 Step 10: the generic discrete state-space form

A broad family of models can be written as

$$h_t = A_t h_{t-1} + B_t x_t, \qquad y_t = C_t h_t. \tag{76}$$

Unrolling yields

$$y_t = \sum_{j \le t} C_t \left(\prod_{i=j+1}^{t} A_i \right) B_j x_j. \tag{77}$$

This is a master template:

- RetNet uses scalar decay matrices;
- GLA uses data-dependent diagonal gates;
- DeltaNet uses correction matrices from the delta rule;
- selective SSMs use input-dependent discretization of a continuous system.

D.12 Step 11: selective state spaces give Mamba

Mamba starts from a continuous-time SSM and makes its discretization input dependent [25]. In simplified form,

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t, \tag{78}$$

where

$$\bar{A}_t = e^{\Delta_t A}, \qquad \bar{B}_t = \left(\int_0^{\Delta_t} e^{(\Delta_t - \tau)A} \, d\tau \right) B(x_t). \tag{79}$$

The step size Δ_t and sometimes the input/output projections depend on the current token.

Interpretation. Earlier linear time-invariant SSMs used the same transition rule everywhere. Mamba lets the model *select* how much to retain or forget based on the current content.

Why this was a breakthrough. It attacked the main weakness of earlier SSMs on language: lack of content-based selection.

D.13 Step 12: structured state-space duality and Mamba-2

Mamba-2 / structured state-space duality (SSD) [36] unifies attention-like operators and SSMs through structured lower-triangular semiseparable matrices.

A first-pass definition. A causal operator is lower triangular because token t may depend only on tokens $j \le t$. It is *semiseparable* when the off-diagonal entries factor in a structured low-rank way. In practical terms, this means a long causal operator can be represented through a small state that evolves across the sequence.

Key derivation. Equation (77) already defines a structured lower-triangular matrix:

$$M_{tj} = C_t \left(\prod_{i=j+1}^{t} A_i \right) B_j, \qquad j \le t. \tag{80}$$

When the transition matrices have the right structure, chunkwise training can be reduced to matrix multiplications that map well to modern accelerators. Mamba-2 refines Mamba with this observation and reports much higher speed.

D.14 Step 13: xLSTM revisits LSTMs with matrix memory

xLSTM revisits the LSTM line at modern scale [29]. The especially relevant variant here is the matrix-memory LSTM (mLSTM), which keeps a matrix state:

$$C_t = f_t C_{t-1} + i_t v_t k_t^\top, \tag{81}$$

$$n_t = f_t n_{t-1} + i_t k_t, \tag{82}$$

$$h_t = o_t \odot \frac{C_t q_t}{\max(1, \ |n_t^\top q_t|)}. \tag{83}$$

This is clearly part of the same finite-state matrix-memory design space as fast weights, linear attention, and retention models.

Why xLSTM matters conceptually. It shows that the old LSTM idea was not "obsolete"; it was waiting for new parameterizations, scaling recipes, and a more expressive memory structure.

D.15 A unifying summary

The modern line can be read as a ladder:

$$\begin{aligned}\text{softmax attention} &\to \text{kernelized attention} \to \text{matrix memory} \to \text{forgetting} \\ &\to \text{data-dependent forgetting} \to \text{delta-rule correction} \to \text{selective SSMs} \\ &\to \text{semiseparable / chunkable structured operators.}\end{aligned}$$

This ladder is the central conceptual object of the survey.

E Recent Influential Works (2023–March 2026)

Learning goals. After this section, you should know which recent works most influenced the field, how they relate to the common framework, what they improved, and what evidence they used to support their claims.

How to read this section. For each work we answer four questions:

1. **Relation to the common framework:** is the work a pure attention baseline, a recurrent/state-space model, or a hybrid?
2. **What it improves:** what specific weakness of earlier models does it target?
3. **Key derivation:** what is the central algebra or algorithm behind the final layer?
4. **Evidence:** what experiments or measurements support the claims?

E.1 Recent work map at a glance

Table 9: Recent influential works and their main contribution.

Work	Family	Main contribution
FlashAttention-2 / 3	Exact attention systems	Raised the practical exact-attention baseline with better GPU utilization.
GQA / MLA	Efficient exact attention	Reduced inference cache cost without removing attention.
RetNet	Recurrent retention	Unified parallel, recurrent, and chunkwise forms.
GLA	Gated linear recurrence	Added data-dependent forgetting with hardware-efficient training.

Work	Family	Main contribution
Mamba	Selective SSM	Added content-dependent state dynamics to make SSMs strong on language.
Mamba-2	Structured operator / SSD	Unified attention and SSMs and improved speed.
DeltaNet	Corrective matrix memory	Replaced naive additive writes by delta-rule correction.
Gated DeltaNet	Corrective + gated memory	Combined targeted correction with learned forgetting.
xLSTM	Matrix-memory LSTM	Scaled LSTM ideas with exponential gating and matrix memory.
Griffin / Hawk	Hybrid recurrence + local attention	Argued for a compromise between pure recurrence and pure attention.
Recurrent Gemma	Open recurrent hybrid	Brought Griffin-style models to open language-model releases.
Samba	Mamba + sliding window attention	Used local exact retrieval plus linear-time long-range summarization.
Jamba / Jamba-1.5	Transformer–Mamba hybrid MoE	Popularized large hybrid open models with long context.
Zamba / Zamba2	Hybrid open models	Used Mamba / Mamba-2 with shared or periodic attention to improve open-weight efficiency.
Hymba	Hybrid-head architecture	Mixed attention heads and SSM heads within the same layer.
Titans	Attention + explicit long-term memory	Separated short-term attention from long-term learned memory.
MiniMax-01	Lightning attention + MoE	Scaled to million-token training context with efficient linear attention.
Kimi Linear	Hybrid delta attention + MLA	Claimed to beat full attention under fair comparisons while reducing cache and improving decoding speed.
Falcon-H1	Hybrid-head family	Showed strong open hybrid-head models across scales and long contexts.

E.2 Recent exact-attention baselines and cache-efficient features

E.2.1 FlashAttention-2

Relation to the common framework. FlashAttention-2 is outside the replacement family: it keeps exact softmax attention [24].

What it improves. Its target is not model expressivity but *hardware efficiency*. It reduces the gap between theoretical and realized performance for exact attention.

Key derivation. The mathematical core is still the online-softmax tiling derivation in Eqs. (28)–(30). FlashAttention-2 changes how this exact algorithm is partitioned across blocks and warps so that modern GPUs spend more time on matrix multiplications and less time on overhead.

Evidence. The paper reports substantially higher model FLOPs utilization and around $2\times$ speedup relative to FlashAttention on A100 GPUs [24].

E.2.2 FlashAttention-3

Relation to the common framework. Again, this is exact attention, not a replacement [46].

What it improves. FlashAttention-3 targets Hopper GPUs specifically. It exploits asynchronous execution and low-precision support.

Key derivation. The exact online-softmax algebra is unchanged. The novelty is a deeper co-design of tiling, overlap of data movement and computation, and low-precision block quantization.

Evidence. The paper reports much higher H100 utilization, up to 740 TFLOPs/s in FP16 and close to 1.2 PFLOPs/s in FP8, together with lower numerical error than a baseline FP8 implementation [46].

E.2.3 GQA

Relation to the common framework. GQA remains exact attention but shrinks the cache size.

What it improves. Compared with full multi-head attention, it reduces decoding bandwidth and cache size. Compared with MQA, it preserves more head diversity.

Key derivation. Replace H separate key-value heads by G groups:

$$\text{cache} \propto nG(d_k + d_v), \qquad 1 < G < H.$$

This preserves head sharing within groups while keeping more expressive power than one shared key-value head.

Evidence. The paper shows that uptrained GQA models approach the quality of full multi-head attention while retaining speed close to MQA [47]. This made GQA one of the most widely adopted attention features in practical LLMs.

E.2.4 MLA via DeepSeek-V2 and DeepSeek-V3

Relation to the common framework. MLA is still attention. Its key change is the *stored state*: compress keys and values into a latent vector per token [33, 39].

What it improves. It aggressively reduces KV cache size and improves throughput for long-context inference.

Key derivation. A simplified latent cache rule is

$$c_t = W_c x_t, \qquad k_t^{(h)} = W_{k,h} c_t, \qquad v_t^{(h)} = W_{v,h} c_t.$$

The cache stores c_t instead of all head-specific keys and values. The attention operation itself remains exact over reconstructed keys and values.

Evidence. DeepSeek-V2 reports large KV cache savings and throughput gains while remaining a top-tier open model; DeepSeek-V3 validated the same MLA architecture at larger scale [33, 39]. MLA quickly became one of the most visible cache-efficiency ideas in the community.

E.3 Recent pure or mostly recurrent replacements

E.3.1 RetNet

Relation to the common framework. RetNet is the fixed-decay special case:

$$S_t = \Lambda S_{t-1} + k_t v_t^\mathsf{T}, \qquad y_t = q_t^\mathsf{T} S_t.$$

What it improves. It directly addresses the old tradeoff between parallel training and fixed-state inference by giving parallel, recurrent, and chunkwise views of the same operator.

Key derivation. Equation (62) shows that fixed recurrent decay becomes a lower-triangular attention-like kernel. That three-way equivalence is the conceptual core of the paper.

Evidence. The paper reports strong scaling behavior and efficient recurrent inference on language modeling benchmarks [22].

E.3.2 Gated Linear Attention (GLA)

Relation to the common framework. GLA is the gated version of retention:

$$S_t = G_t \odot S_{t-1} + k_t v_t^\top.$$

What it improves. Its target is the weakness of fixed decay. Some information should decay quickly; some should persist. A learned gate allows token-dependent forgetting.

Key derivation. Equation (65) shows that the effective weight on an old write is a product of future gates. So the model learns a content-dependent causal mask.

Evidence. The paper reports competitive language modeling against LLaMA-style Transformers and strong length generalization, together with a hardware-efficient training implementation called FlashLinearAttention [23].

E.3.3 Mamba

Relation to the common framework. Mamba is a selective SSM:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t.$$

What it improves. It addresses the main weakness of earlier SSMs on language: lack of content-based selection.

Key derivation. The key step is input-dependent discretization:

$$\bar{A}_t = e^{\Delta_t A}, \qquad \bar{B}_t = \left(\int_0^{\Delta_t} e^{(\Delta_t - \tau)A} \, d\tau \right) B(x_t).$$

The current token changes the state transition itself.

Evidence. The paper reports strong results across language, genomics, and audio, plus faster inference than comparable Transformers in the authors' experiments [25].

E.3.4 Mamba-2 / structured state-space duality

Relation to the common framework. Mamba-2 is the strongest theoretical unification in this line. It treats attention-like kernels and SSMs as neighboring structured causal operators [36].

What it improves. It improves both theory and systems performance. Conceptually, it explains why SSMs and attention are not separate worlds. Practically, it yields faster kernels than the original Mamba.

Key derivation. The structured lower-triangular matrix

$$M_{tj} = C_t \left(\prod_{i=j+1}^{t} A_i \right) B_j$$

is the bridge between recurrence and causal operator matrices. With suitable structure, chunked training becomes a sequence of matrix multiplications.

Evidence. The paper reports 2–8× faster kernels than Mamba while maintaining competitive language-model quality [36].

E.3.5 DeltaNet

Relation to the common framework. DeltaNet is the correction branch of matrix memory.

What it improves. It tries to reduce interference between old and new writes and to improve retrieval-style behavior.

Key derivation. The essential update is

$$S_t = S_{t-1} + \eta_t (v_t - r_t) k_t^{\mathsf{T}}, \qquad r_t = S_{t-1} k_t.$$

The memory first predicts, then writes only the correction.

Evidence. The 2024 scaling paper trained 1.3B models and reported better perplexity and downstream zero-shot performance than strong linear-recurrent baselines such as Mamba and GLA in the reported setting [30].

E.3.6 Gated DeltaNet

Relation to the common framework. Gated DeltaNet is the natural fusion of GLA and DeltaNet.

What it improves. It combines rapid forgetting of stale memory with targeted correction of the active memory content.

Key derivation. Use Eq. (75). Gating decides which memory survives; the delta rule updates the surviving memory in the key direction.

Evidence. The paper reports gains over Mamba-2 and DeltaNet on language modeling, commonsense reasoning, in-context retrieval, and length extrapolation [42].

E.3.7 xLSTM

Relation to the common framework. xLSTM comes from the LSTM branch but lands squarely in the matrix-memory design space through mLSTM.

What it improves. It asks whether LSTM ideas can compete again when scaled with modern training recipes, normalization, and richer memories.

Key derivation. The matrix-memory equations in Eq. (83) replace the scalar LSTM cell by a key–value matrix plus a normalization state.

Evidence. The paper reports favorable scaling and competitive language-model performance against modern state-space and Transformer baselines [29].

E.4 Hybrids: the increasingly dominant compromise

E.4.1 Griffin and Hawk

Relation to the common framework. Hawk is a gated linear recurrent model; Griffin adds local attention [26]. So Griffin sits exactly at the interface between fixed-state recurrence and explicit token retrieval.

What it improves. It targets the weakness of pure recurrence on exact recall while keeping fast long-context inference.

Key derivation. A schematic layer is

$$h_t^{\mathrm{rec}} = \mathcal{R}(h_{t-1}^{\mathrm{rec}}, x_t), \qquad h_t^{\mathrm{loc}} = \mathrm{Attn}(x_t; x_{t-w:t}), \qquad y_t = W[h_t^{\mathrm{rec}}; h_t^{\mathrm{loc}}]. \tag{84}$$

The recurrent path summarizes the long past; local attention handles high-resolution recent recall.

Evidence. The paper reports that Hawk beats the reported Mamba results on several downstream tasks, while Griffin matches Llama-2 with far fewer training tokens and extrapolates to longer sequences than seen during training [26].

E.4.2 RecurrentGemma

Relation to the common framework. RecurrentGemma is a production-scale, open Griffin-family model [27].

What it improves. Its importance is practical rather than theoretical: it brought recurrent hybrids into widely available open models.

Key derivation. Architecturally it inherits the Griffin recipe of gated recurrence plus local attention.

Evidence. The paper reports performance comparable to similarly sized Gemma baselines while using a fixed-size recurrent state and fewer training tokens [27].

E.4.3 Samba

Relation to the common framework. Samba alternates Mamba layers with sliding-window attention [32]. It is a direct hybrid of selective state spaces and local exact retrieval.

What it improves. It aims to combine unlimited-context linear-time recurrence with exact recent-memory recall.

Key derivation. A schematic layer schedule is

$$\text{Mamba block} \leftrightarrow \text{sliding-window attention block}.$$

This exploits the complementary strengths of the two mixers rather than forcing one to do everything.

Evidence. The paper reports strong performance up to million-token contexts and large throughput gains over grouped-query Transformers at long prompts [32].

E.4.4 Jamba and Jamba-1.5

Relation to the common framework. Jamba interleaves Transformer blocks and Mamba blocks and adds mixture-of-experts capacity [31, 35]. It is one of the clearest large-scale statements that hybrids are a practical default.

What it improves. The model tries to keep the exact retrieval power of attention, the state efficiency of Mamba, and the capacity benefits of MoE.

Key derivation. The main design object is the block schedule, not a single new equation:

$$\text{attention block} \rightarrow \text{Mamba block} \rightarrow \text{MoE feed-forward as configured}.$$

This is important conceptually: the frontier question shifted from *best single mixer* to *best mixture of mixers*.

Evidence. The papers report strong open-model performance, high throughput, and long-context results up to 256K tokens [31, 35].

E.4.5 Zamba and Zamba2

Relation to the common framework. Zamba uses a Mamba backbone with a shared attention module; Zamba2 upgrades the design with Mamba-2 style blocks [34, 38].

What they improve. They pursue a parameter-efficient open-weight hybrid strategy: use mostly recurrent/state-space computation but insert enough attention to preserve strong language-model quality.

Key derivation. A simple schematic view is

$$h^{\ell+1} = h^{\ell} + \mathrm{SSM}(h^{\ell}) + \mathcal{A}_{\text{shared or periodic}}(h^{\ell}).$$

The exact share pattern differs by model family, but the guiding idea is minimal attention for maximal recall benefit.

Evidence. The technical reports present strong open-weight results and substantial inference efficiency gains for their model classes [34, 38].

E.4.6 Hymba

Relation to the common framework. Hymba hybridizes inside the layer itself by using attention heads and SSM heads in parallel [37].

What it improves. The paper argues that hybridization should happen at the head level, not only by alternating whole layers.

Key derivation. A simplified head-level form is

$$y_t = \mathrm{Concat}(y_t^{\text{attn-heads}}, y_t^{\text{ssm-heads}})W_O. \tag{85}$$

The same layer can therefore perform exact recall and long-horizon summarization simultaneously.

Evidence. The paper reports strong small-model performance, major cache reduction, and large throughput gains against public baselines in its size range [37].

E.4.7 Falcon-H1

Relation to the common framework. Falcon-H1 is another hybrid-head family combining attention and SSM-style components in parallel [44].

What it improves. It targets efficiency across a wide scale range while keeping strong open-model quality.

Key derivation. The family again follows the head-parallel hybrid principle of mixing exact and compressed-context channels in one architecture.

Evidence. The report presents multiple open models, long-context support up to 256K, and strong benchmark results across scales [44].

E.5 Memory-augmented and million-context systems

E.5.1 Titans

Relation to the common framework. Titans is not just a recurrent replacement. It adds a separate long-term memory module that learns at test time, while attention acts as short-term memory [40].

What it improves. Instead of forcing one mechanism to solve both short-term exact recall and long-term persistent storage, Titans separates those roles.

Key derivation. A simplified formulation is

$$h_t^{\text{short}} = \text{Attn}(x_t; x_{t-w:t}), \qquad m_t = \mathcal{M}(m_{t-1}, x_t), \qquad y_t = \mathcal{G}(h_t^{\text{short}}, m_t), \tag{86}$$

where $\mathcal{M}$ is a learned long-term memory update and $\mathcal{G}$ merges the short- and long-term branches.

Evidence. The paper reports gains on language, reasoning, genomics, and time series, together with effective scaling to contexts above 2M tokens in long-retrieval tests [40].

E.5.2 Lightning Attention and MiniMax-01

Relation to the common framework. Lightning Attention remains in the linear-attention family but focuses on a hardware-friendly, block-structured implementation [45]. MiniMax-01 scales that family with MoE to very long contexts [41].

What they improve. The target is the training and inference cost of extremely long contexts.

Key derivation. Lightning Attention splits the computation into

- **intra-block interactions,** handled by local exact attention;
- **inter-block interactions,** handled by linear-attention-style state updates.

That avoids the cumulative-sum bottleneck that otherwise hurts practical linear-attention throughput.

Evidence. The Lightning Attention paper reports strong algorithmic speed properties; MiniMax-01 reports million-token training context, multi-million-token inference extrapolation, and strong large-model performance at those context lengths [41, 45].

E.5.3 Kimi Linear

Relation to the common framework. Kimi Linear is the clearest late-2025 synthesis of the finite-state branch with efficient exact-attention features [43]. Its Kimi Delta Attention (KDA) extends Gated DeltaNet, and the full architecture hybridizes KDA with MLA layers.

What it improves. The paper claims that a carefully designed linear/hybrid attention architecture can outperform full attention under fair comparisons while reducing cache and improving long-context decoding speed.

Key derivation. Conceptually the progression is

$$\text{DeltaNet} \rightarrow \text{Gated DeltaNet} \rightarrow \text{KDA}$$

where KDA uses finer-grained gating and a specialized diagonal-plus-low-rank chunkwise algorithm. At the architecture level, KDA layers are mixed with periodic MLA layers, so the full model combines finite-state corrective memory with compressed exact attention.

Evidence. The paper reports that a 48B-total / 3B-activated hybrid outperforms a full-MLA baseline under the same recipe, reduces KV cache by up to 75%, and reaches up to 6× decoding throughput at 1M context [43].

E.6 What changed in the community from 2023 to 2026?

The 2023–2026 period changed the field in three important ways.

1. **Exact attention remained extremely strong.** FlashAttention-2 and -3 kept improving the exact baseline.
2. **Cache efficiency became a first-class design target.** GQA and MLA changed practical long-context deployment.

3. **Hybrids became normal.** Griffin, RecurrentGemma, Samba, Jamba, Zamba, Hymba, Titans, MiniMax-01, Kimi Linear, and Falcon-H1 all mix multiple memory mechanisms.

This is why modern surveys should not frame the field as "Transformer versus non-Transformer." The real design space is *mixtures of explicit retrieval and compressed memory*.

References

[1] Jürgen Schmidhuber. Learning Complex, Extended Sequences Using the Principle of History Compression. *Neural Computation*, 4(2):234–242, 1992.

[2] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997.

[3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. In *ICLR*, 2015.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *NeurIPS*, 2017.

[5] Noam Shazeer. Fast Transformer Decoding: One Write-Head Is All You Need. arXiv:1911.02150, 2019.

[6] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating Long Sequences with Sparse Transformers. arXiv:1904.10509, 2019.

[7] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The Long-Document Transformer. arXiv:2004.05150, 2020.

[8] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for Longer Sequences. In *NeurIPS*, 2020.

[9] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The Efficient Transformer. In *ICLR*, 2020.

[10] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-Attention with Linear Complexity. arXiv:2006.04768, 2020.

[11] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers Are RNNs: Fast Autoregressive Transformers with Linear Attention. In *ICML*, 2020.

[12] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. HiPPO: Recurrent Memory with Optimal Polynomial Projections. In *NeurIPS*, 2020.

[13] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A Nyström-Based Algorithm for Approximating Self-Attention. In *AAAI*, 2021.

[14] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Łukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking Attention with Performers. In *ICLR*, 2021.

[15] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.

[16] Ofir Press, Noah A. Smith, and Mike Lewis. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. arXiv:2108.12409, 2021.

[17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*, 2022.

[18] Albert Gu, Karan Goel, and Christopher Ré. Efficiently Modeling Long Sequences with Structured State Spaces. In *ICLR*, 2022.

[19] Daniel Y. Fu, Tri Dao, Khaled Kamal Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry Hungry Hippos: Towards Language Modeling with State Space Models. In *ICLR*, 2023.

[20] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena Hierarchy: Towards Larger Convolutional Language Models. In *ICML*, 2023.

[21] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran G V, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, and Jian Zhu. RWKV: Reinventing RNNs for the Transformer Era. In *Findings of EMNLP*, 2023.

[22] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive Network: A Successor to Transformer for Large Language Models. arXiv:2307.08621, 2023.

[23] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated Linear Attention Transformers with Hardware-Efficient Training. arXiv:2312.06635, 2023.

[24] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691, 2023.

[25] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752, 2024.

[26] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. arXiv:2402.19427, 2024.

[27] Aleksandar Botev and the Griffin, RLHF, and Gemma Teams. RecurrentGemma: Moving Past Transformers for Efficient Open Language Models. arXiv:2404.07839, 2024.

[28] Zhen Qin, Songlin Yang, Bailin Wang, and Yoon Kim. HGRN2: Gated Linear RNNs with State Expansion. arXiv:2404.07904, 2024.

[29] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended Long Short-Term Memory. In *NeurIPS*, 2024.

[30] Songlin Yang, Bailin Wang, Yikang Shen, Yoon Kim, and Mohit Bansal. Parallelizing Linear

Transformers with the Delta Rule over Sequence Length. arXiv:2406.06484, 2024.

[31] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A Hybrid Transformer–Mamba Language Model. arXiv:2403.19887, 2024.

[32] Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling. arXiv:2406.07522, 2024.

[33] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434, 2024.

[34] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, Anna Golubeva, Vasudev Shyam, James Whittington, Jonathan Pilault, and Beren Millidge. Zamba: A Compact 7B SSM Hybrid Model. arXiv:2405.16712, 2024.

[35] Jamba Team. Jamba-1.5: Hybrid Transformer–Mamba Models at Scale. arXiv:2408.12570, 2024.

[36] Tri Dao and Albert Gu. Transformers Are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. In *ICML*, 2024.

[37] Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. Hymba: A Hybrid-head Architecture for Small Language Models. arXiv:2411.13676, 2024.

[38] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, Anna Golubeva, Vasudev Shyam, James Whittington, Jonathan Pilault, and Beren Millidge. The Zamba2 Suite: Technical Report. arXiv:2411.15242, 2024.

[39] DeepSeek-AI. DeepSeek-V3 Technical Report. arXiv:2412.19437, 2024.

[40] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to Memorize at Test Time. arXiv:2501.00663, 2025.

[41] MiniMax, Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, Enwei Jiao, Gengxin Li, Guojun Zhang, Haohai Sun, Houze Dong, Jiadai Zhu, Jiaqi Zhuang, Jiayuan Song, Jin Zhu, Jingtao Han, Jingyang Li, Junbin Xie, Junhao Xu, Junjie Yan, Kaishun Zhang, Kecheng Xiao, Kexi Kang, Le Han, Leyang Wang, Lianfei Yu, Liheng Feng, Lin Zheng, Linbo Chai, Long Xing, Meizhi Ju, Mingyuan Chi, Mozhi Zhang, Peikai Huang, Pengcheng Niu, Pengfei Li, Pengyu Zhao, Qi Yang, Qidi Xu, Qiexiang Wang, Qin Wang, Qiuhui Li, Ruitao Leng, Shengmin Shi, Shuqi Yu, Sichen Li, Songquan Zhu, Tao Huang, Tianrun Liang, Weigao Sun, Weixuan Sun, Weiyu Cheng, Wenkai Li, Xiangjun Song, Xiao Su, Xiaodong Han, Xinjie Zhang, Xinzhu Hou, Xu Min, Xun Zou, Xuyang Shen, Yan Gong, Yingjie Zhu, Yipeng Zhou, Yiran Zhong, Yongyi Hu, Yuanxiang Fan, Yue Yu, Yufeng Yang, Yuhao Li, Yunan Huang, Yunji Li, Yunpeng Huang, Yunzhi Xu, Yuxin Mao, Zehan Li, Zekang Li, Zewei Tao, Zewen Ying, Zhaoyang Cong, Zhen Qin, Zhenhua Fan, Zhihang Yu, Zhuo Jiang, and Zijia Wu. MiniMax-01: Scaling Foundation Models with Lightning Attention. arXiv:2501.08313, 2025.

[42] Songlin Yang, Bailin Wang, Yikang Shen, Yoon Kim, and Mohit Bansal. Gated Delta Networks: Improving Mamba2 with Delta Rule. arXiv:2412.06464, 2024.

[43] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi Linear: An Expressive, Efficient Attention Architecture. arXiv:2510.26692, 2025.

[44] Jingwei Zuo, Maksim Velikanov, Ilyas Chahed, Younes Belkada, Dhia Eddine Rhayem, Guillaume Kunsch, Hakim Hacid, Hamza Yous, Brahim Farhat, Ibrahim Khadraoui, Mugariya Farooq, Giulia Campesan, Ruxandra Cojocaru, Yasser Djilali, Shi Hu, Iheb Chaabane, Puneesh Khanna, Mohamed El Amine Seddik, Ngoc Dung Huynh, Phuc Le Khac, Leen AlQadi, Billel Mokeddem, Mohamed Chami, Abdalgader Abubaker, Mikhail Lubinets, Kacper Piskorski, and Slim Frikha. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance. arXiv:2507.22448, 2025.

[45] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. Lightning Attention-2: A Free Lunch for Handling Unlimited Sequence Lengths in Large Language Models. arXiv:2401.04658, 2024.

[46] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision. arXiv:2407.08608, 2024.

[47] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *EMNLP*, 2023.